University of Central Punjab

E-Doorstep Services

info@edoorstepservices.com

BSCS FINAL PROJECT

E-DoorStep Services

Project Advisor

Prof. Tanveer Ahmed

Presented by:
Group ID: 2

Registration No:	**Name:**
S3F15BSCS0004	**Ahmed Idrees**
S3F15BSCS0018	**Rida Shafiq**
S3F15BSCS0022	**Madiha Akram**

Faculty of Information Technology

University of Central Punjab

Software Requirements Specification

Version 2

E-DoorStep Services

Group ID: 2

Member Name	Primary Responsibility
1. Ahmed Idrees	PHP + APP Coding
2. Rida Shafiq	Documentation + XML files.
3. Madiha Akram	Design + Diagram.

Table of Contents

1. Introduction 7
1.1. Product 7
1.2. Scope 7
1.3. Business Goals 8
1.4. Document Conventions 9
1.5. References 9
2. Overall Description 10
2.1. Product Features 10
2.2. User Classes and Characteristics 11
Fig.1 Users 12
2.3. Operating Environment 12
2.4. Design and Implementation Constraints 13
Fig.2 Gantt chart 14
2.5. Assumptions and Dependencies 14
3. Functional Requirements 15
3.1. Use-Case Descriptions 16
Fig.3 Use Case 1 16
Fig.4 Use Case 2 17
Fig.5 Use Case 3 18
Fig.6 Use Case 4 19
Fig.7 Use Case 5 20
Fig.8 Use Case 6 20
Fig.9 Use Case 7 21
Fig.10 Use Case 8 22
Fig.11 Use Case 9 23
Fig.12 Use Case 10 23
Fig.13 Use Case 11 24
Fig.14 Use Case 12 25
Fig.15 Use Case 13 26
Fig.16 Use Case 14 26
Fig.17 Use Case 15 27
4. Nonfunctional Requirements 28
4.1. Performance Requirements 28
4.2. Safety Requirements: 28
4.3. Security Requirements 29

4.4. Software Quality Attributes 29
5. Other Requirements: 30
5.1. Physical Implementation Requirements: 30
Appendix A: Glossary 32
Appendix B: Analysis Models 34
7.1. Use Case Diagrams: 34
Fig.18 Use Case Diagram 1 34
Fig.19 Use Case Diagram 2 35
Fig.20 Use Case Diagram 3 36
Fig.21 Use Case Diagram 4 37
Fig.22 Use Case Diagram 5 38
7.2. Class diagram: 39
Fig.23 Class Diagram 40
7.3. Sequence Diagrams: 41
Fig.24 Sequence Diagram 1 41
Fig.25 Sequence Diagram 2 42
Fig.26 Sequence Diagram 3 43
Fig.27 Sequence Diagram 4 44
Fig.26 Sequence Diagram 5 45
Fig.27 Sequence Diagram 6 46
Fig.28 Sequence Diagram 7 47
Fig.29 Sequence Diagram 8 47
Fig.30 Sequence Diagram 9 48
7.4. Entity Relationship Diagram: 49
Fig.31 ERD 49
Appendix C: Design Models 50
Fig.32 Architecture Diagram 50
Fig.33 Tier Diagram 51
Appendix D: Screenshots 52
Welcome slides 52
Welcome slide and login Activity 53
Signup Activity & Password Reset Activity 54
Home Activity & Profile Activity 55
Contact Us Page 56
Services & Individual service Activity 57
Booking and Ordered items 58

About us and Terms & Condition 59
Admin Dashboard 60
Organization Dashboard 60
Appendix F: IV & V Report 61

1. Introduction

1.1. Product

We have encountered the problem of repairs for different household, office and work station things like wooden, electricity, and sanitary. People have to call the repairers by going to their shops and then they come according to their time. In this scenario of delay process people faces frustration. So our goal is to solve this problem by providing an online booking system through Android Mobile App. We will register relevant trusted **organization** and assign repair tasks to them. As a result people will be able to easily book the repairer and get the repair services at their doorstep within the given time.

This is **E-DoorStep Service**. The aim of E-Doorstep Services is to provide the home, office or work station services to users at their place. Using **mobile app** users can submit bookings for different available services. Users can also make contact with the **admin** in case of any query. Our admin will manage all the bookings and assign them to organizations those are registered with us. On organization **dashboard** they can handle their workers details, manage bookings and update booking status. We'll provide this services in Sialkot city.

1.2. Scope

The scope of our project covers three sides with respect to stake holders i.e. **Admin, Organization and customer.**
The **scope of admin** section is that it has a login dashboard, where they can perform some specific function:

- Admin can add the organization.
- Admin can edit the organization information.
- Admin can delete the organization.
- Admin can assign the booking to organization.
- Admin can view the registered customer.
- Admin will receive the contact us details by the customer.
- Admin can add, edit or delete the services.
- Admin can delete the booking.

The **scope of organization** section is that it also has login dashboard, where they can perform some specific functions:

- Organization can add the worker.
- Organization can edit the worker.
- Organization can delete worker.
- Organization can assign booking to their worker.

- Organization can update the booking status to either completed or processing.

The **scope of the customer** section is that first of all customers are divided into two categories i.e. as guest, as registered user.

Customer as guest booking:

- Customer can view the home page, where they can access the services, about us, contact us, terms & conditions but it in order to access the Booking section it has to first register.
- Customer can add the services to the **cart**.
- Customer can view the cart.
- Customer can also remove items from the cart.
- Customer can proceed to booking by clicking on the book now button and fill the form, then submit.

Registered customer:

- The registered customer can do all of the above mentioned functions with an extra feature which is that is can also view the Booking details.
- Login customer will also logout.
- In case of forget password customer need to reset password.

Also we will mention our policies in the terms and condition section of our app.

1.3. Business Goals

The project goal is to provide people online bookings for repairing and maintenance of electricity, sanitary and wooden goods etc. Customer will select required services and mention repair date according to their need and submit booking, then admin will check the details. Then admin will assign booking to one of registered organization with us and they will send their worker.

Worker will visit the customer place for repairing and complete the job and notify the organization which will then updated on admin side and customer side.

- Here’re following of our main objectives:
- Provide different repair services at one place.
- Provide best services to our customers with protecting their rights according to our terms and conditions.
- Provide timely services without any delay.
- Provide maximum customer satisfaction.

1.4. Document Conventions

Here are some standards we have used in this document:

- We have used Times New Roman font style. Because it is world's go-to font for official looking documents.
- And font size is 14.
- We have used heading styles for specifying the table of contents.
- And bold the important words and sub headings.
- We have used tables where necessary to increase the readability, organization of data and for comparison

1.5. References

We have consulted many online tutorials through these websites:

1. https://www.w3schools.com/css/default.asp
2. https://www.w3schools.com/php/default.asp
3. https://www.w3schools.com/html/default.asp
4. https://developer.android.com/training/basics/firstapp
5. http://www.1202performance.com/atricles/how-to-write-performance-requirements-with-example/
6. https://www.android-examples.com/volley-post-request-php-mysql/
7. https://www.simplifiedcoding.net/retrieve-data-mysql-database-android/

2. Overall Description

2.1. Product Features

1. Related to the Customer Area

- **Sign Up**

Customer would have to register with E-Doorstep App to book services.

- **Login**

Username and password that allows a person to login into app.

- **Services**

Consumer's view of services that include its details.

- **Add to cart**

Customer can add desire service or services into cart to book.

- **Booking**

Customer can book the cart services with providing details in booking form.

- **Contact us**

User can contact us in case of any query or problem.

- **Security**

User passwords are saved in Database in MD5 format.

2. Related to the Admin Area

• Login

Username and password that allows admin to log in to system.

• Services Management

1. Add service
2. Update/edit service
3. Delete service

• Organization Management

1. Add organization
2. Update/edit organization
3. Delete organization

• User Management

1. View Users

• Booking Management

1. Assign booking to organization
2. Delete the booking

3. Related to the Organization Area

• Login

Username and password that allows organization to log in to system.

• Worker Management

1. Add worker
2. Update/edit worker
3. Delete worker

• Booking Management

1. Update booking status. (Complete, Processing)
2. Assign booking to worker

2.2. User Classes and Characteristics

As we have described in the product description section that our project is for office, household and work stations.
So our user classes are based on household, office users and work station users.

Household user and their characteristic:

- The household users are the common people. They might or might not be educated.
- But our user interface is very friendly that even uneducated person can also use it.

Because of the simplicity, use of images and user guidance.

- We have used simple wording and icons that can be easily understood by the native users.

Office user and their characteristic:

- Office users are the educated persons.
- They can easily understand our policies, working procedure.

Work stations user and their characteristic:

- Work station users are also educated person.
- Work stations usually refer to the manufacturing departments.

We also have two categories of users that we have defined in our project which are:

- Users that use our app as guest.
- Users that use our app as registered persons.

Both these classes have some privileges.

Guest user	Register user
1. They would view the Welcome slides.	They would view the Welcome slides
2. Will not access to signup.	Will sign up for first use.
3. Click on guest booking and view home page but can't view profile.	Login and view home page and profile.
4. Can access services, contact us, about us and terms & conditions.	It can access services, order details contact us, about us and terms & conditions.
5. Can use add to cart feature.	Can use add to cart feature.
6. Can do Booking by filling form.	Can do Booking by filling form.
7. Will have to pay 300 service charges.	Will have to pay 200 service charges.
8. No logout interface.	Can logout from the account.

Fig.1 Users

2.3. Operating Environment

1. Hardware and Software Specification:

Below is a list of the minimum Hardware and Software requirements to access E-DoorStep Services:

Operating System:

- Windows 7, Windows 8 or Windows 10 for admin and organizations
- Android mobile running 5.0 Lollipop and above
- Compatible browsers for admin and organizations activities

2. Hardware Requirements:

It should run on every standard pc/laptop/android phones/tablet currently in use.

2.4. Design and Implementation Constraints

For the designing and implementation of the this software we required the following resources and our Gantt chart illustrates that how many time is required to deliver this software with the description of the phases.

Resources Required
Tools and technologies used with reasoning:

Tools:

- Android Studio IDE will be used for mobile app development. Android Studio is official IDE from Google for building android apps free.
- Notepad++ will be used to write codes for website admin panel and organization panel. It is free and open source software and works well for web app development.
- We need Xamp Local Host for local system development.
- We'll make MySQL Database to store data. MySQL is a freely available open source Relational Database Management System (RDBMS) that uses Structured Query Language (SQL). SQL is used for adding, accessing and managing content in a database.

Technologies:

- We will use Java programming Language for coding in Android Studio IDE. Java is widely use language and perfect for making android apps.
- We'll use HTML, CSS, JS and jQuery for scratching admin and organization panel layout.
- PHP will be used for backside programming to our admin panel. PHP is consider to be best and easy language when it comes to make website.

Timing constraints:
Project Plan / Schedule:
Our project is planned according to our supervisor and group member mutual cooperation. We will thoroughly go through each phase for completing each and every single step. The work is break down into the activities and we've shown the time duration require to complete the project via Gant Chart.

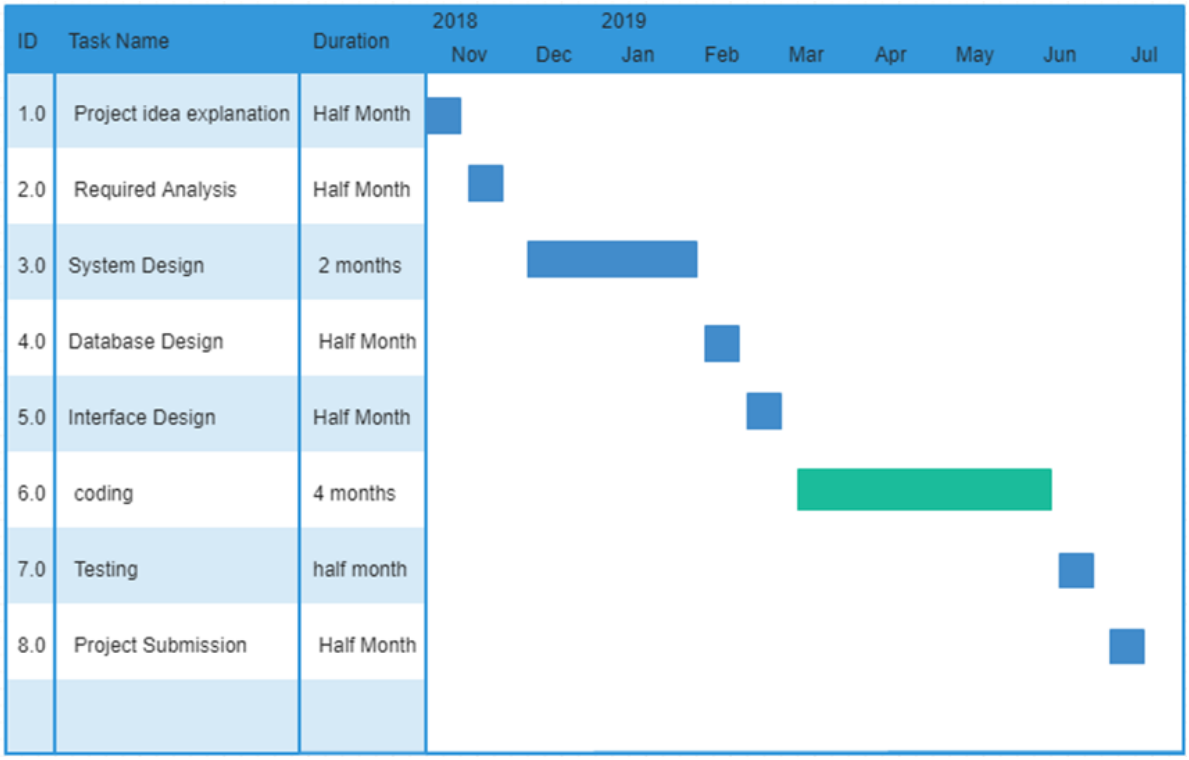


Fig.2 Gantt chart

2.5. Assumptions and Dependencies

- Android Phone or tablet is required with lollypop 5.0 or above.
- Internet connection is required for booking.
- Illiterate people can find difficulties with booking.
- Our website is host on cPanel. In case of server down or maintenance problem our services will be suspended.
- In case of DNS update, site will be off for some minutes.
- There may be many obstacles in the correct integration of the admin panel and user interface i.e. mobile app. Our system highly dependent on this integration.
- New updated browsers required to run admin and organizations panel.

3. Functional Requirements

Functional requirements are the requirements which a system provides to its end users. There are three modules in this phase.

1. **Admin module**
   - Login
   - Services Management
   - User Management
   - Organization Management
   - Booking Management
2. **Customer module**
   - Sign Up
   - Login
   - Services
   - Product Detail
   - Cart
   - Booking
   - Contact us
3. **Organization module**
   - Login
   - Worker Management
   - Booking Management
   - Contact with admin
   - Contact with customer

3.1. Use-Case Descriptions

Table 1: Signup

Identifier	**Sign up**	
Purpose	Customer needs to create his/her account.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer should be able to access signup page. Customer should fill the form and submit. Account should be generated.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will open edoorstepapp.	System will display login page.
2	Customer will click on create account.	System will display signup form.
3	Customer will fill the form.	System will respond.
4	Customer will click on submit.	System will create the account.
[1]Alternate Course of Action		
S#	Actor Action	System Response
1	Edoorstep app might not open.	System might have some error.
2	Click button might not work.	System might have hanged.
3	Form might not be filled correctly.	System has check on form fields.
4.	If form not submitted.	System will refresh the form.

Fig.3 Use Case 1

Table 2: Login

Identifier	**Login**	
Purpose	Customer needs to login.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer must access login page. Customer should fill the login form and submit. Access granted.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will open edoorstepapp.	System will display login page.
2	Customer will fill the login form.	System will respond.
3	Customer will click on Login.	System will open home page.
[2]Alternate Course of Action		
S#	Actor Action	System Response
1	Edoorstep app might not open.	System might have some error.
3	Form might not be filled correctly.	System has check on form fields.
4.	If form not submitted.	System will refresh the form.

Fig.4 Use Case 2

Table 3: Service Management

Identifier	**Service Management**
Purpose	Admin will manage the services.
Pre-conditions	System should be in running state and working properly.

Post-conditions	Admin will add the service. Admin will edit the service. Admin will delete the service.	
Typical Course of Action		
S#	Actor Action	System Response
1	Admin will add the service.	System will be updated.
2	Admin will edit the service.	System will be updated.
3	Admin will delete the service.	System will be updated.
[3]Alternate Course of Action		
S#	Actor Action	System Response
1	Service might not add.	System will display error message.
3	Service might not edit.	System will display error message.
4.	Service might not be deleted.	System will display error message.

Fig.5 Use Case 3

Table 4: Worker Management

Identifier	**Worker Management**	
Purpose	Organization will manage the Workers.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Organization will add the worker. Organization will edit the worker. Organization will delete the worker. Organization will assign bookings to worker.	
Typical Course of Action		
S#	Actor Action	System Response
1	Organization will add the worker.	System will be updated.

2	Organization will edit the worker.	System will be updated.
3	Organization will delete the worker.	System will be updated.
4	Organization will assign booking to worker.	System will be updated.
[4]Alternate Course of Action		
S#	Actor Action	System Response
1	Worker might not be added.	System will display error message.
2	Worker might not be edited.	System will display error message.
3	Worker might not be deleted.	System will display error message.
4.	Booking may not be assigned.	System might have some error.

Fig.6 Use Case 4

Table 5: Booking Management

Identifier	**Booking Management**	
Purpose	Admin will handle the bookings	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Admin will assign booking to organization. Admin will delete booking.	
Typical Course of Action		
S#	Actor Action	System Response
1	Admin will assign booking to Organization.	System will be updated.
2	Admin will delete booking.	System will be updated.
[5]Alternate Course of Action		
S#	Actor Action	System Response

1	Booking might not be assigned to Organization.	System will display error message.
2	Booking might not be deleted.	System will display error message.

Fig.7 Use Case 5

Table 6: Organization Management

Identifier	**Organization Management**	
Purpose	Admin will manage the Organization.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Admin will add the Organization. Admin will edit the Organization. Admin will delete the Organization.	
Typical Course of Action		
S#	Actor Action	System Response
1	Admin will add the Organization.	System will be updated.
2	Admin will edit the Organization.	System will be updated.
3	Admin will delete the Organization.	System will be updated.
[6]Alternate Course of Action		
S#	Actor Action	System Response
1	Organization might not be added.	System will display error message.
2	Organization might not be edited.	System will display error message.
3	Organization might not be deleted.	System will display error message.

Fig.8 Use Case 6

Table 7: View Services

Identifier	**View Services**	
Purpose	Customer will view the services.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer will view wooden services. Customer will view sanitary services. Customer will view electricity services.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will view Wooden services.	System will response.
2	Customer will view Sanitary services.	System will response.
3	Customer will view Electricity services.	System will response.
[7]Alternate Course of Action		
S#	Actor Action	System Response
1	Wooden services might not be viewed.	System might have some error.
2	Sanitary services might not be viewed.	System might have some error.
3	Electricity services might not be viewed.	System might have some error.

Fig.9 Use Case 7

Table 8: Select Services

Identifier	**Select Service**
Purpose	Customer will select the service.
Pre-conditions	System should be in running state and working properly.

Post-conditions	Customer will select wooden services. Customer will select sanitary services. Customer will select electricity services.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will select Wooden services.	System will be Updated
2	Customer will select Sanitary services.	System will be Updated.
3	Customer will select Electricity services.	System will be Updated.
[8]Alternate Course of Action		
S#	Actor Action	System Response
1	Wooden services might not be selected.	System will display error message.
2	Sanitary services might not be selected.	System will display error message.
3	Electricity services might not be selected.	System will display error message.

Fig.10 Use Case 8

Table 9: Add to Cart

Identifier	**Add to cart**	
Purpose	Customer will select the service and add to cart.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer will add to cart one service. Customer will add to cart more than one service.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will add to cart one service.	System will respond.
2	Customer will add to cart more than one service.	System will respond.

[9]Alternate Course of Action		
S#	Actor Action	System Response
1	Service might not be added to cart.	System might have some error or problem.
2	More than one services might not be added to cart.	System might have some error or problem.

Fig.11 Use Case 9

Table 10: View Cart

Identifier	**View cart**	
Purpose	Customer will view cart.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer will click on cart. Customer will view services added to the cart.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will click on cart.	System will respond.
2	Customer will view services added to the cart.	System will display cart.
[10]Alternate Course of Action		
S#	Actor Action	System Response
1	Customer might not be able to view cart.	System might have some error or problem.
2	Customer might not find added services to the cart.	System might have some error or problem.

Fig.12 Use Case 10

Table 11: Booking Placement

Identifier	**Booking/Order Placement**	
Purpose	Customer will place booking for services.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer must access booking form. Customer will fill the booking form and submit it.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will open booking form.	System will display the booking form.
2	Customer will fill the form.	System will respond.
3	Customer will click on submit button.	System will respond.
[11]Alternate Course of Action		
S#	Actor Action	System Response
1	Booking form might not open.	System might have some error.
2	Form might not be filled correctly.	System has check on form fields.
3	If booking form not submitted.	System will refresh the form.

Fig.13 Use Case 11

Table 12: Booking Management (Organization)

Identifier	Booking Management	
Purpose	Organization will manage the booking.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Organization will view bookings. Organization will assign bookings to workers. Organization will update booking status.	
Typical Course of Action		
S#	Actor Action	System Response
1	Organization will view bookings.	System will respond.

2	Organization will assign bookings to workers.	System will be updated.
3	Organization will update booking status.	System will be updated.
[12]Alternate Course of Action		
S#	Actor Action	System Response
1	Bookings might not be viewed by the organization.	System might have some error.
2	Organization might not assign booking to the worker.	System will display error message.
3	Organization might not be updated booking status.	System might have some error.

Fig.14 Use Case 12

Table 13: Contact with Customer

Identifier	**Contact with customer.**	
Purpose	Admin will contact with the customer.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Admin will get phone No and email from customer database. Admin will contact to the customer through phone No and email.	
Typical Course of Action		
S#	Actor Action	System Response
1	Admin will get phone No from database.	System will respond.
2	Admin will get Email from database.	System will respond.
3	Admin will call or send message by email.	System will respond.
[13]Alternate Course of Action		
S#	Actor Action	System Response

1	Phone No might not be found by admin.	System might have some error or problem.
2	Email might not be found by admin.	System might have some error or problem.
3	Call or email might not be sent to the customer.	System might have some error or problem.

Fig.15 Use Case 13

Table 14: Contact with Admin

Identifier	**Contact with Admin**	
Purpose	Customer will contact with Admin.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer will access contact us page. Fill the contact us form and submit that form.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will open Contact Us form.	System will display the form.
2	Customer will fill the Contact Us form.	System will respond.
3	Customer will click on submit button.	System will respond.
[14]Alternate Course of Action		
S#	Actor Action	System Response
1	Contact us form might not open.	System might have some error or problem.
2	Form might not be filled correctly.	System has check on form fields.
3	If form not submitted.	System will refresh the form.

Fig.16 Use Case 14

Table 15: logout

Identifier	**Logout**	
Purpose	Customer needs to Logout.	
Pre-conditions	System should be in running state and working properly.	
Post-conditions	Customer must click on menu bar. Customer will find logout button from menu. Customer will click on logout button to logout.	
Typical Course of Action		
S#	Actor Action	System Response
1	Customer will click on menu bar.	System will display menu.
2	Customer will click on logout button from menu.	System will show login page.
[15]Alternate Course of Action		
S#	Actor Action	System Response
1	Customer may not find menu bar.	System might have some error.
3	Customer might not be able to logout the system after clicking on logout button.	System will display error message.

Fig.17 Use Case 15

4. Nonfunctional Requirements

4.1. Performance Requirements

In order to assess the performance of a system the following must be clearly specified:
• Response Time
• Workload
• Scalability

Response Time:
Response time is the time in which the system should response the user. It is very important factor. Because the users don't use the apps that take more time to load.
Our app will provide response from 0.1 second to 4 second of maximum. But we will maintain the consistency of the response time.

Work Load:
Work load is the number of users supported by the system at a time. Our system supports many users at a time, because we will use volley library which uses asynchronous calls and it forms request queue in case of multiple users thus handling the deadlock situation. In Volley library the requests are added into queue which leads to automatic scheduling of network requests.

Scalability:
In one respect scalability is simply specified as the increase in the system's workload that the system should be able to process. We have Linux cPanel hosting that supports upto 10k of bookings. In future if this limits is near to approach we can also increase it.

4.2. Safety Requirements:

Software System Safety optimizes system safety in the design, development, use, and maintenance of software systems and their integration with safety critical hardware systems in an operational environment.
There are three major types of software failure. Not all of these can be rectified by the programmers. The three types are:

- **Software logic errors**
- **Software support errors**
- **Hardware failures**

Software logic errors are often a result of the programmer making errors in the coding, whether it is simply a mistake on their part, or an incorrect set of requirements they are following. In addition, it may have been a mistake made in the design phase which follows through into the implementation of the system.
We will use android studio that does not run the program in case of the bugs. But to ensure the logic error free system we will make one module and test it to check the logic and implementation.

Software support errors are linked to the software being used to create the program. Perhaps these errors are from the compiler, an external library being used, or even the programming language.
Due to the fact that software needs **hardware** to run on, software safety also has to take hardware safety into account. Therefore, hardware failure is an important aspect to consider in safety critical systems. Our app need maximum of 15MB space of storage and will be develop for android users.

4.3. Security Requirements

We have use some security measures to protect the data of our customer, for authentication as well as the requests that are being sent to the server.

1. **Data Protection:** we have used SSL certificate to prevent our data from any third party.
2. **Email Authentication:** we send email to our customer on the basis of different events such as Signup, contact us and booking.
3. Only one account can be registered on one email.
4. **User authentication:** we have used complete user authentication mechanism on password reset request.

4.4. Software Quality Attributes

Following are some software quality attributes that we will take into consideration while developing our system:

Correctness:
Application should be correct in terms of its functionality, calculations used internally and the navigation should be correct. This means application should adhere to functional requirements. We will make sure to follow all the functional requirements. Our system will satisfy the needs.

Ease of use & learn:
We make our system easy to use and learn. For fulfilling this requirement we will make our procedure as minimum as possible. And all the steps will be mentioned. Use of interactive images will make our system easy to use and understandable by the user.

Reliability:
Measure if product is reliable enough to sustain in any condition. Should give consistently correct results. System should behave error free. We will perform proper testing to ensure the quality of our product to be error free.

Integrity or Security
Integrity comes with security. System integrity or security should be sufficient to prevent unauthorized access to system functions, preventing information loss, ensure that the software is protected from virus infection, and protecting the privacy of data entered into the system.We have already describe the measures to ensure integrity in security requirement section.

Interoperability
Interoperability of one system to another should be easy for product to exchange data or services with other systems. Different system modules should work on different operating system platforms, different databases and protocols conditions. Our mobile app is integrated with the web and database.

5. Other Requirements:

5.1. Physical Implementation Requirements:

For the physical implementation of our system there were multiple tools and technologies were used. Which are as follows:

Tools & Technology
XAMPP
XAMPP is an open-source web server package that works on various platforms. It is actually an acronym with X meaning "cross" platform, A for Apache HTTP server, M for MySQL, P for PHP, XAMPP helps webpage developers, programmers, and designers check and review their work using their computers even without connection to the web or internet. It can also be used to create and configure with databases written in MySQL and/or SQLite. And since XAMPP is designed as a cross-platform server package.

PHP
PHP will be used for backside programming to our admin panel. PHP is a general-purpose scripting language that was originally designed for web development to produce dynamic web pages. PHP will be embedded in an HTML document to enable the document to behave dynamically (reacting to user inputs and other changes).

MySQL
we'll make MySQL Database. MySQL is a freely available open source Relational Database Management System (RDBMS) that uses Structured Query Language (SQL). SQL is used for adding, accessing and managing content in a database. It is most noted for its quick processing, proven reliability, ease and flexibility of use.

Photoshop
An image editing software developed and manufactured by Adobe Systems Inc. We'll use it to design banners, photos and mockups of website and mobile app.

Android Studio
Android Studio IDE will be used for mobile app coding. Android Studio is official IDE from Google for building android apps free.

Java
We will use Java Programming Language for coding in Android Studio IDE. Java is widely use language and perfect for making android apps.

HTML, CSS, JS, jQuery
We'll use HTML, CSS and JS for scratching admin layout. jQuery is a JavaScript library and will use for handling compatibility.

Domain/Hosting:
We will use "edoorstepservices" and cPanel hosting.

Launch on app store:
We will launch our app on play store for android.
Below is a list of the minimum Hardware and Software requirements to access E-DoorStep Services:

Operating System:
- Windows 7, Windows 8 or Windows 10 for admin
- Android mobile running 5.0 Lollipop and above
- Compatible browsers for admin activities

Appendix A: Glossary

Abbreviations & Terms	**Meanings**
SRS:	A software requirements specification (SRS) is a detailed description of a software system to be developed with its functional and non-functional requirements.
E-Doorstep Service:	E for Electronic meaning through online. Title of our project.
Mobile App:	A **mobile app** is a software **application** developed specifically for use on small, wireless computing devices, such as smartphones and tablets, rather than desktop or laptop computers.
Web Base application:	A **web-based application** is any program that is accessed over a network connection using HTTP, rather than existing within a device's memory.
Admin:	**Admin** is the activity or process of organizing an institution or organization.in our case the owner of our project.
Organization:	Organization are those that will provide the repairers to go to customer places and repair.
Workers:	Workers are described as the repairers. They are the part of the organization module.
Scope:	Scope means setting boundaries. The extent of the area or subject matter that something deals with or to which it is relevant.
Guest Customers:	The customers without login.
Cart:	A facility on a website that records items selected by a customer for book until the booking is completed.
Timely:	Done or occurring at a favorable or useful time
Household:	Household the natives of Sialkot.
Work Stations:	Work stations refers to the manufacturing places
Office:	Any office users can also book repair with us.
Interface:	A device or program enabling a user to communicate with a computer. We are using graphical user interface.
IDE:	Integrated Development Environment) A set of programming tools for writing applications (source code editor, compiler, debugger, etc.
Use Case Diagram:	A use case is a series of related interactions between a user (and more generally an “actor”) and a system that enables the user to achieve a goal.

System Sequence Diagram:	A sequence diagram is an interaction diagram. From the name it is clear that the diagram deals with some sequences, which are the sequence of messages flowing from one object to another.
ERD:	Entity Relationship Diagram, also known as **ERD**, ER Diagram or ER model, is a type of structural diagram for use in database design.
Architecture Diagram:	An **architecture diagram** is a graphical representation of a set of concepts, that are part of an **architecture**, including their principles, elements and components.
Test Cases:	A **TEST CASE** is a set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly.
Class Diagram:	Class diagrams are the most common diagrams used in UML. Class diagram consists of classes, interfaces, associations and collaboration. Class diagrams basically represent the object oriented view of a system which is static in nature.

Appendix B: Analysis Models

7.1. Use Case Diagrams:

Use Case Diagram Related to Admin Side:

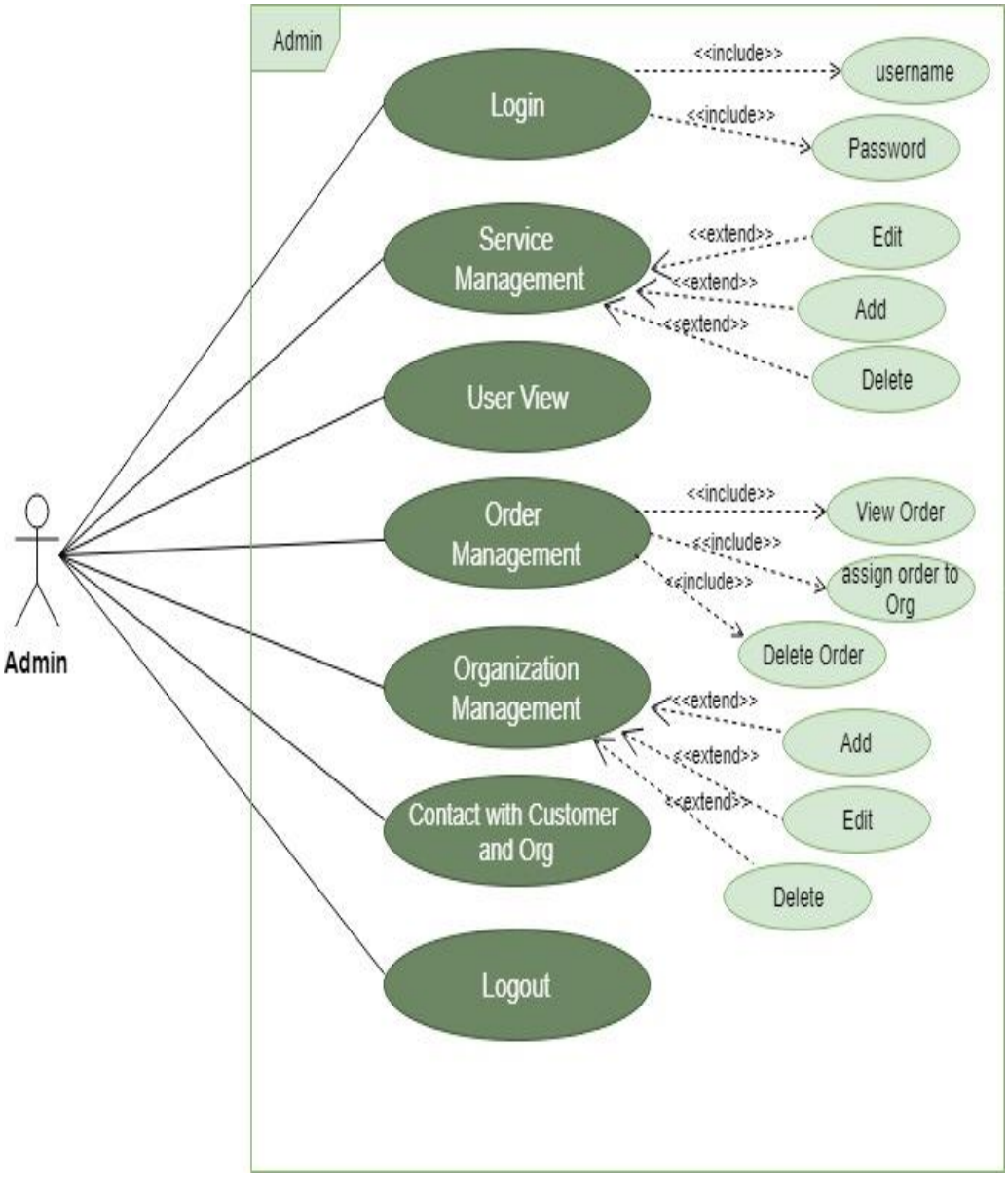


Fig.18 Use Case Diagram 1

Use Case Diagram Related to Customer side (Registered)

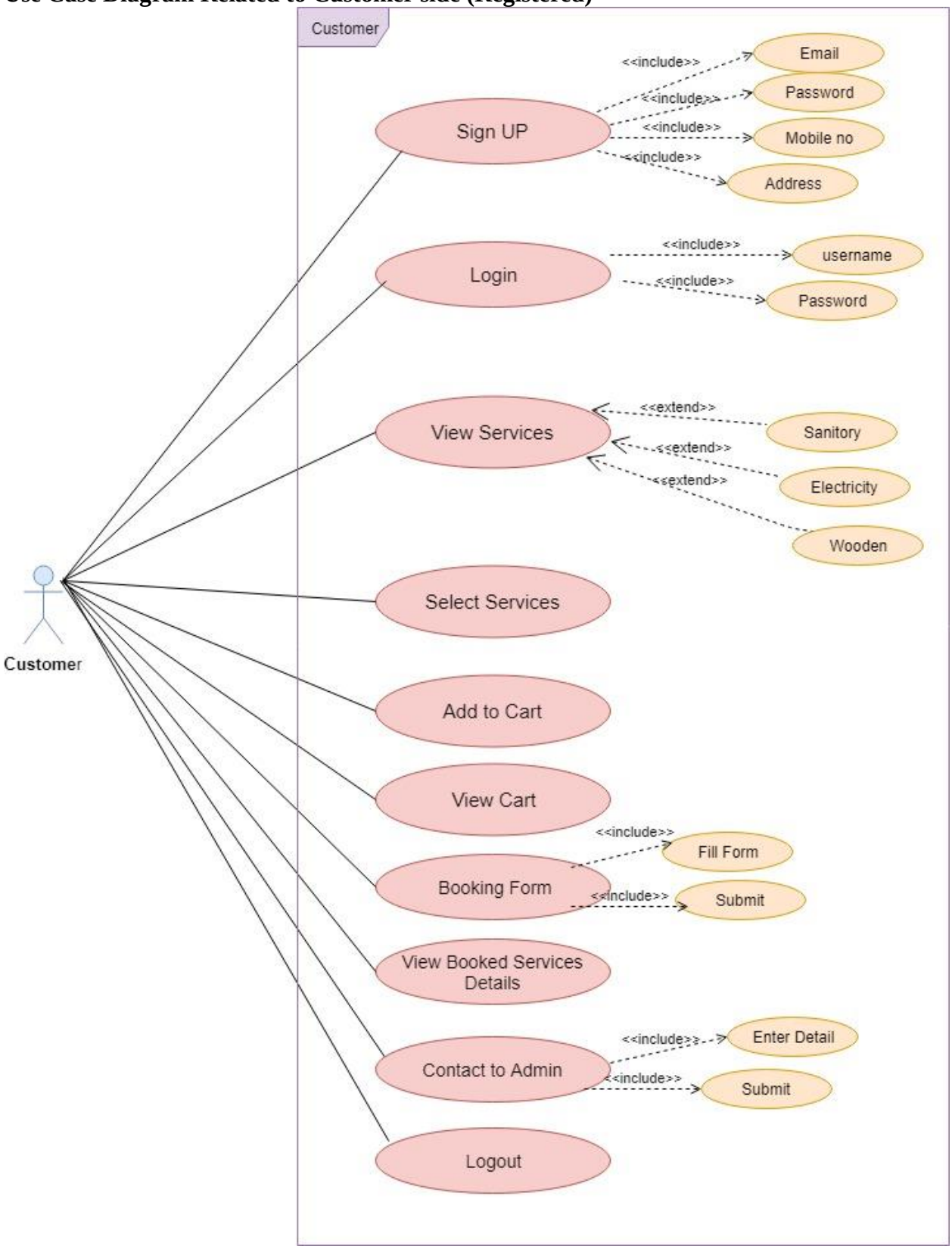


Fig.19 Use Case Diagram 2

Use Case Diagram Related to Customer side (Guest)

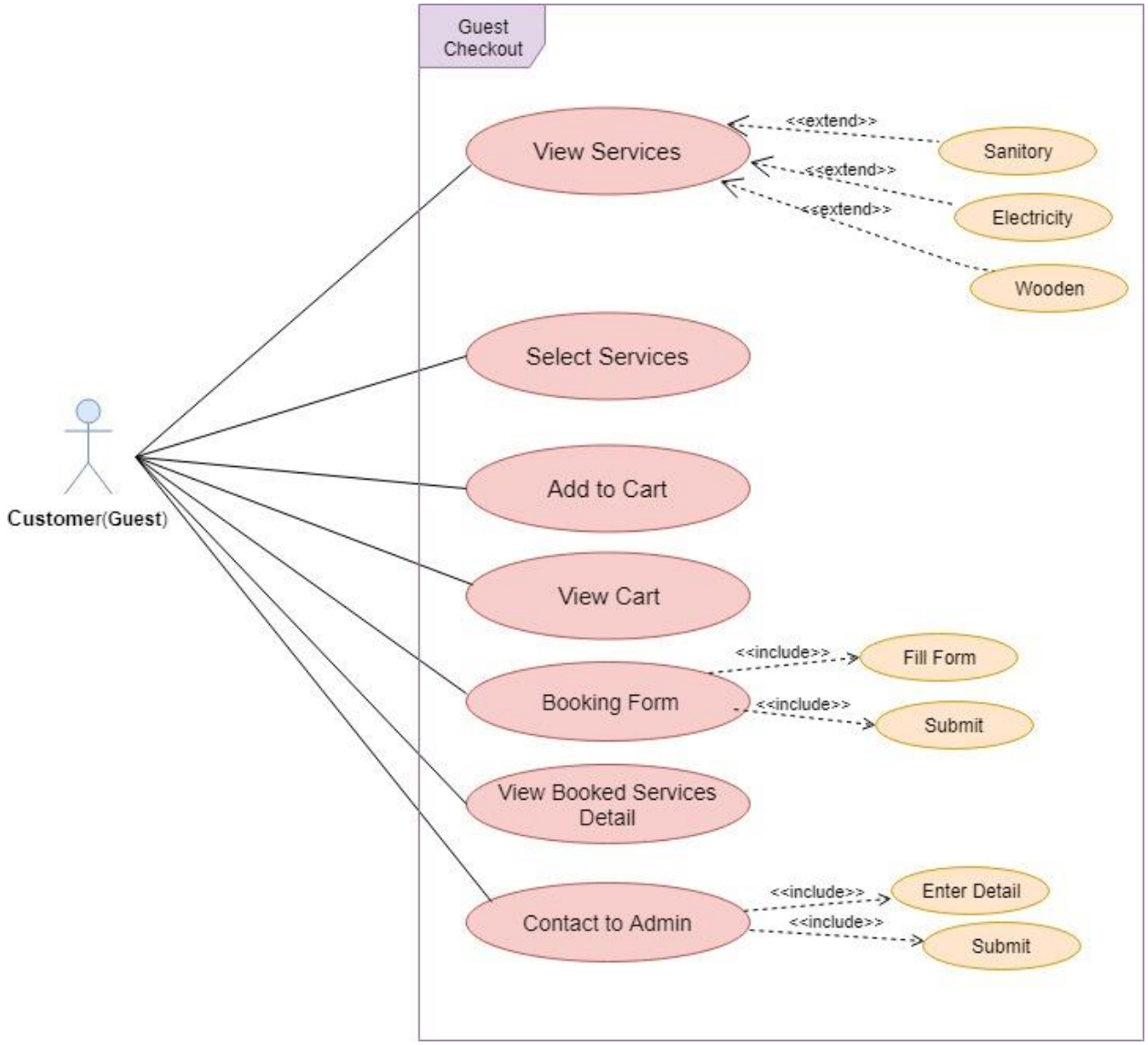


Fig.20 Use Case Diagram 3

Use Case Diagram Related to Organization side

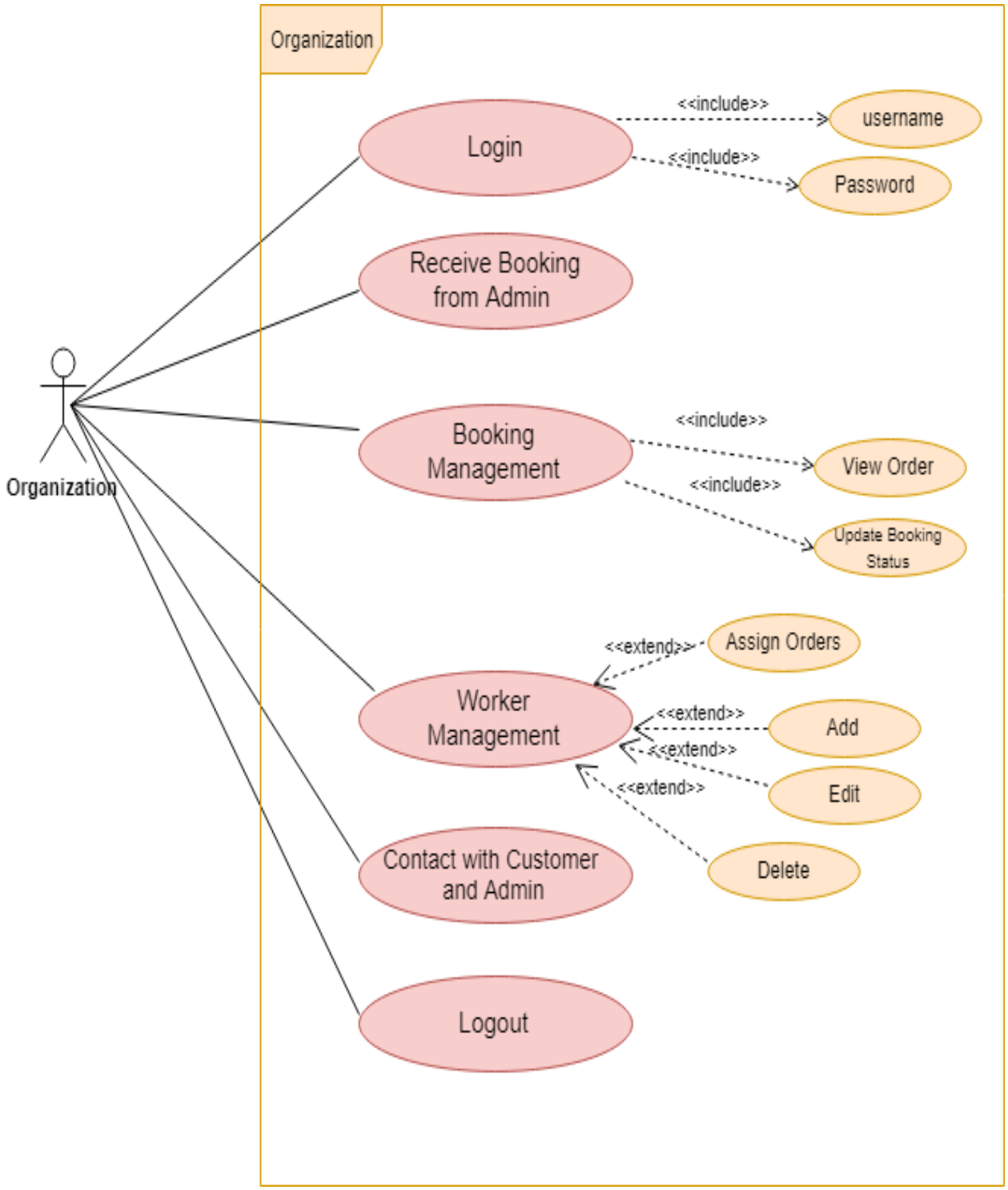


Fig.21 Use Case Diagram 4

Use Case for whole system

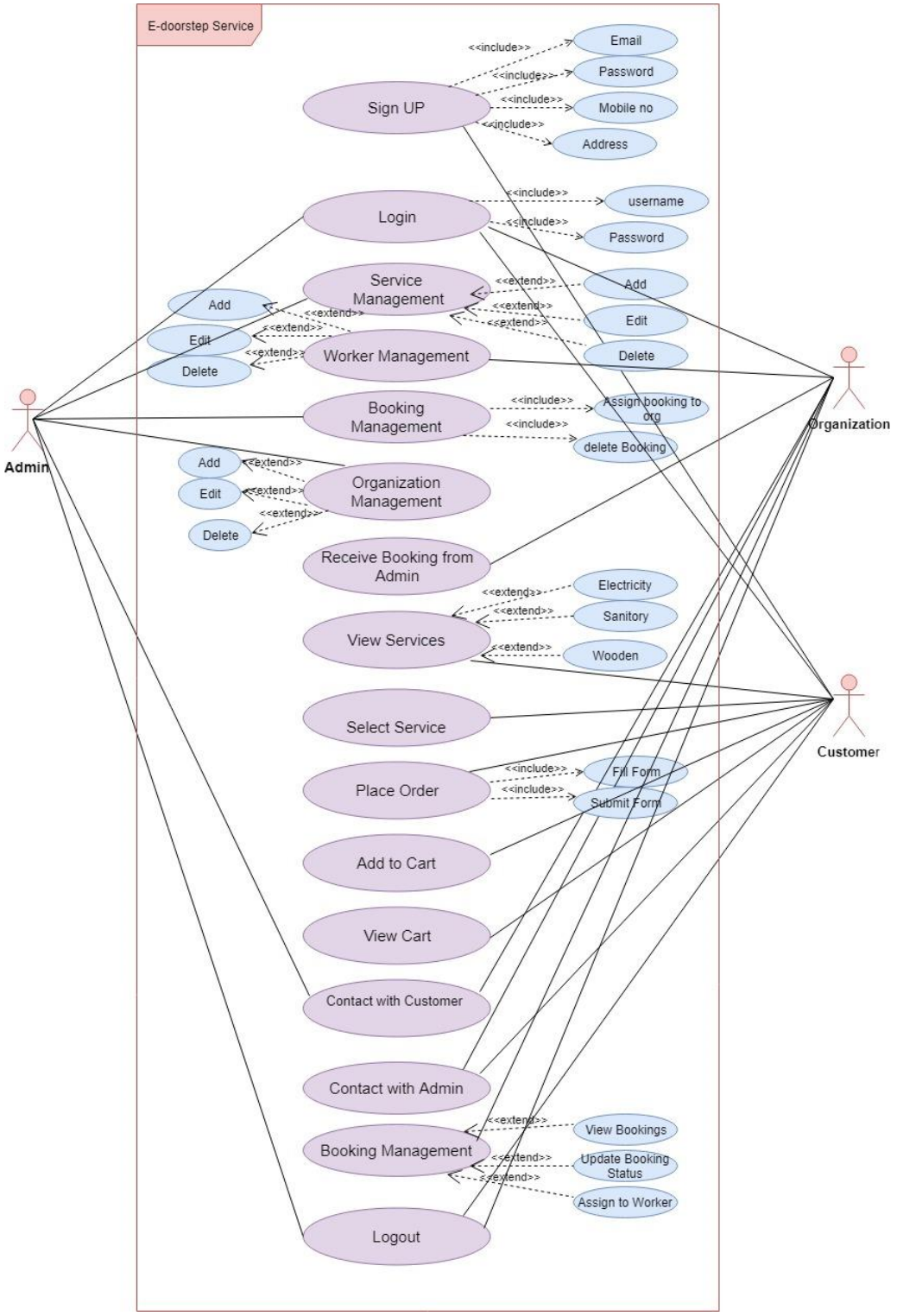


Fig.22 Use Case Diagram 5

7.2. Class diagram:

Class diagrams are the most common diagrams used in UML. Class diagram consists of classes, interfaces, associations and collaboration. Class diagrams basically represent the object oriented view of a system which is static in nature. Active class is used in a class diagram to represent the concurrency of the system. Class diagram represents the object orientation of a system. So it is generally used for development purpose. This is the most widely used diagram at the time of system construction.

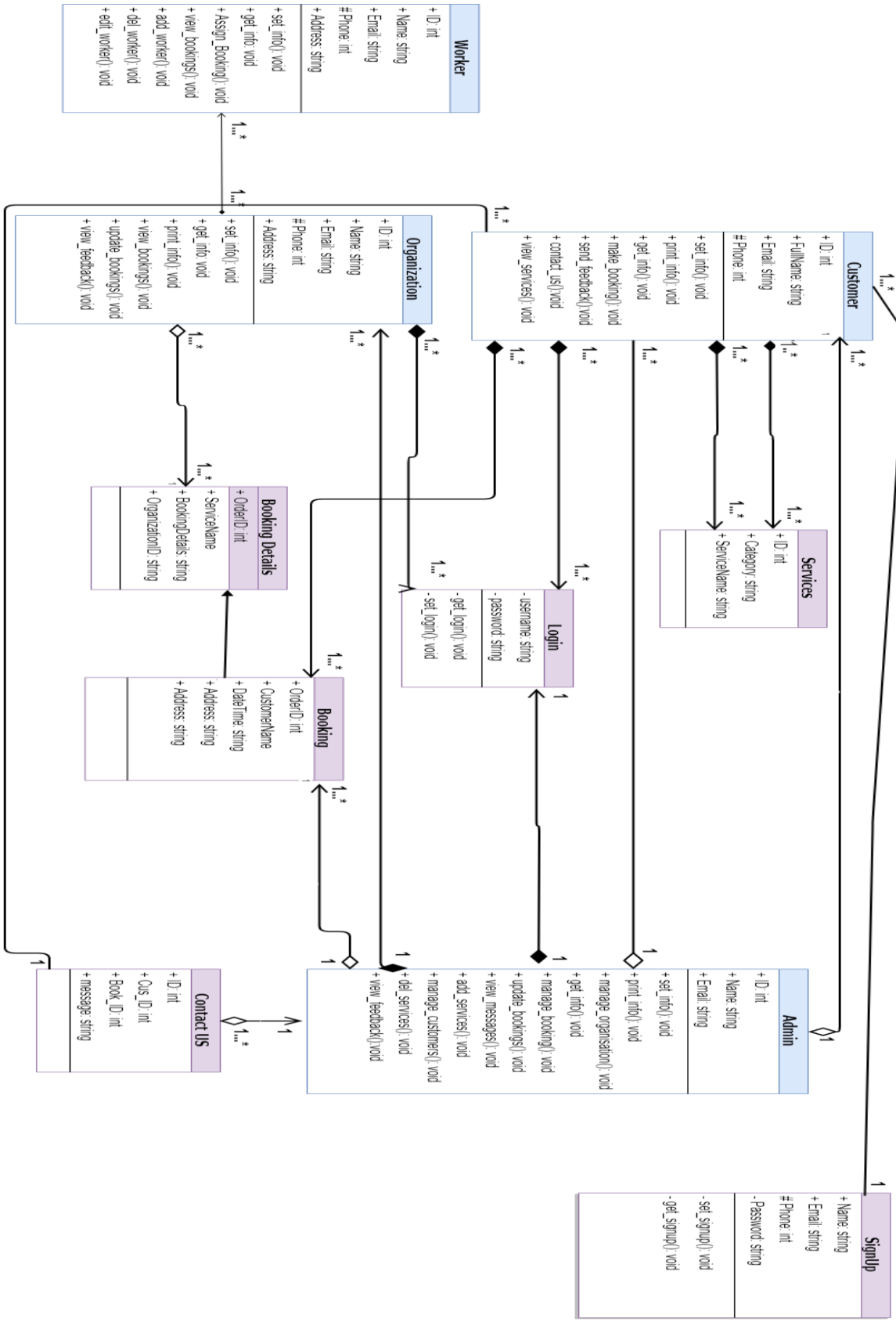


Fig.23 Class Diagram

7.3. Sequence Diagrams:

A sequence diagram is an interaction diagram. From the name it is clear that the diagram deals with some sequences, which are the sequence of messages flowing from one object to another. Interaction among the components of a system is very important from implementation and execution perspective. So Sequence diagram is used to visualize the sequence of calls in a system to perform a specific functionality.

Admin Login

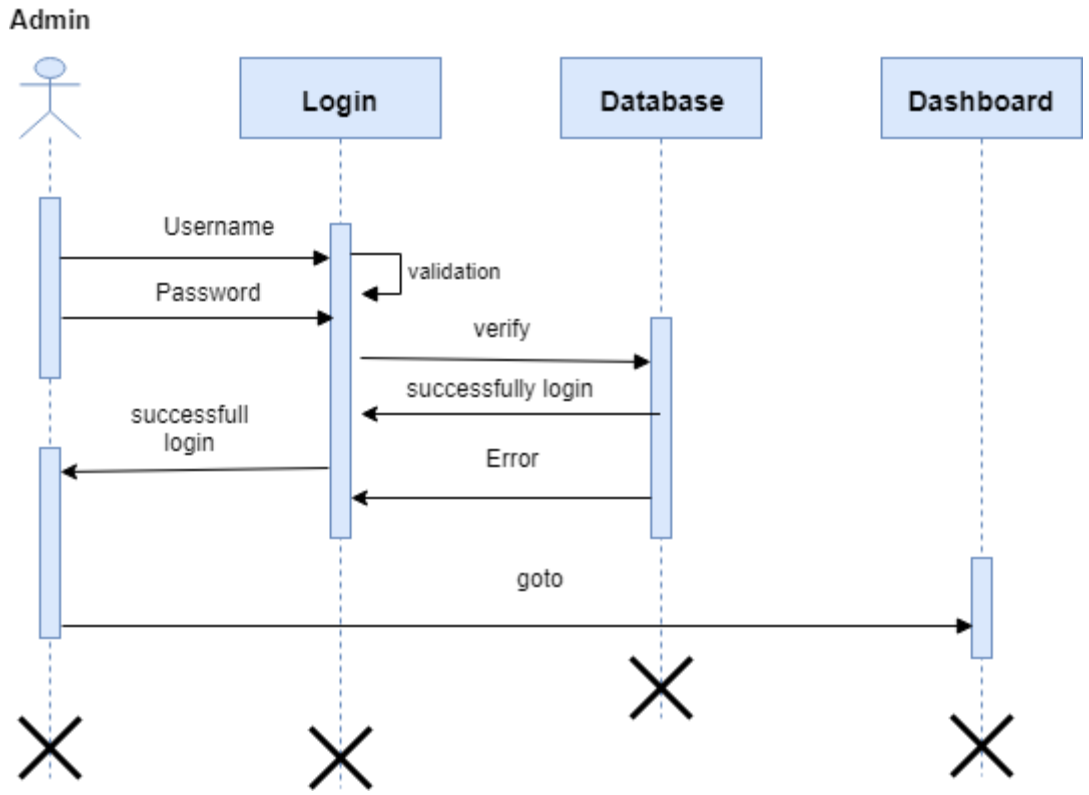


Fig.24 Sequence Diagram 1

Admin Working:

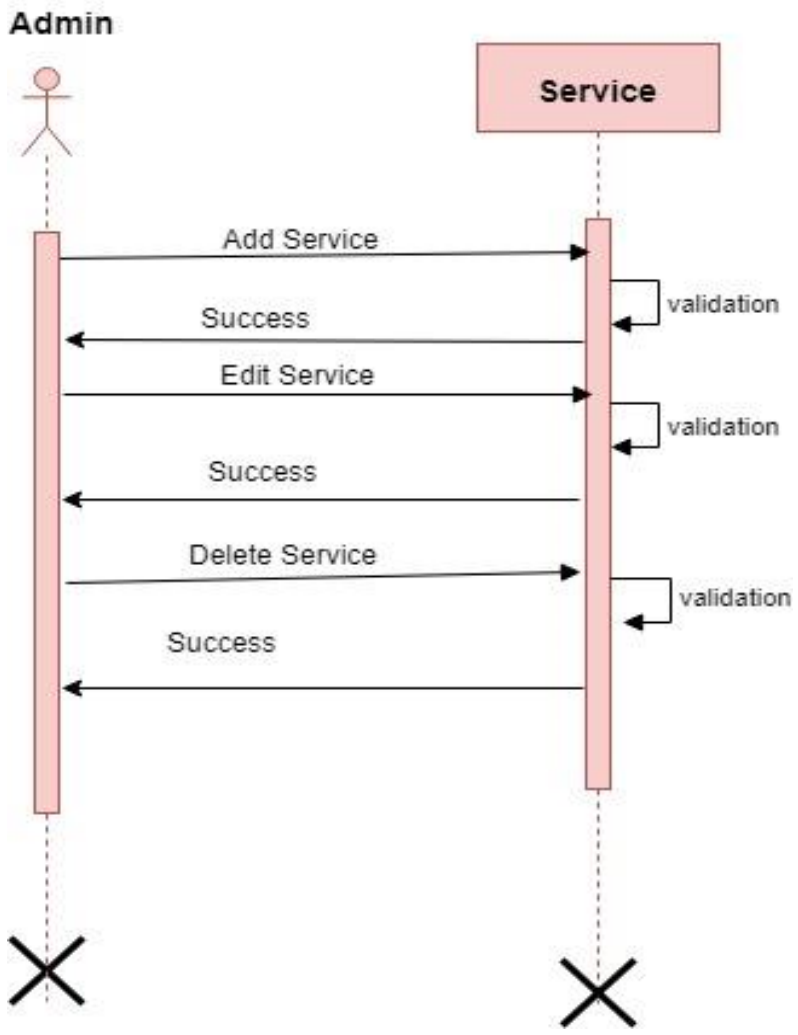


Fig.25 Sequence Diagram 2

Admin Working with organization:

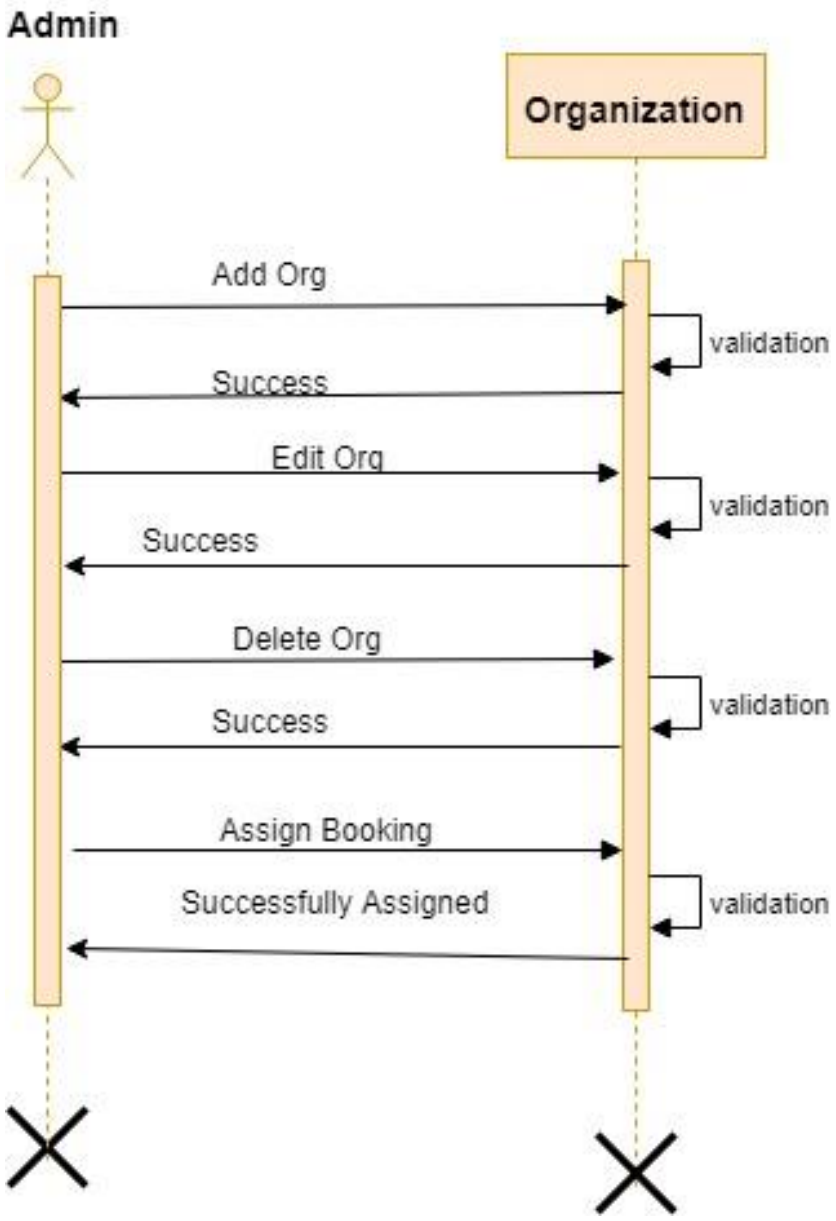


Fig.26 Sequence Diagram 3

Relation between organization, admin and customer:

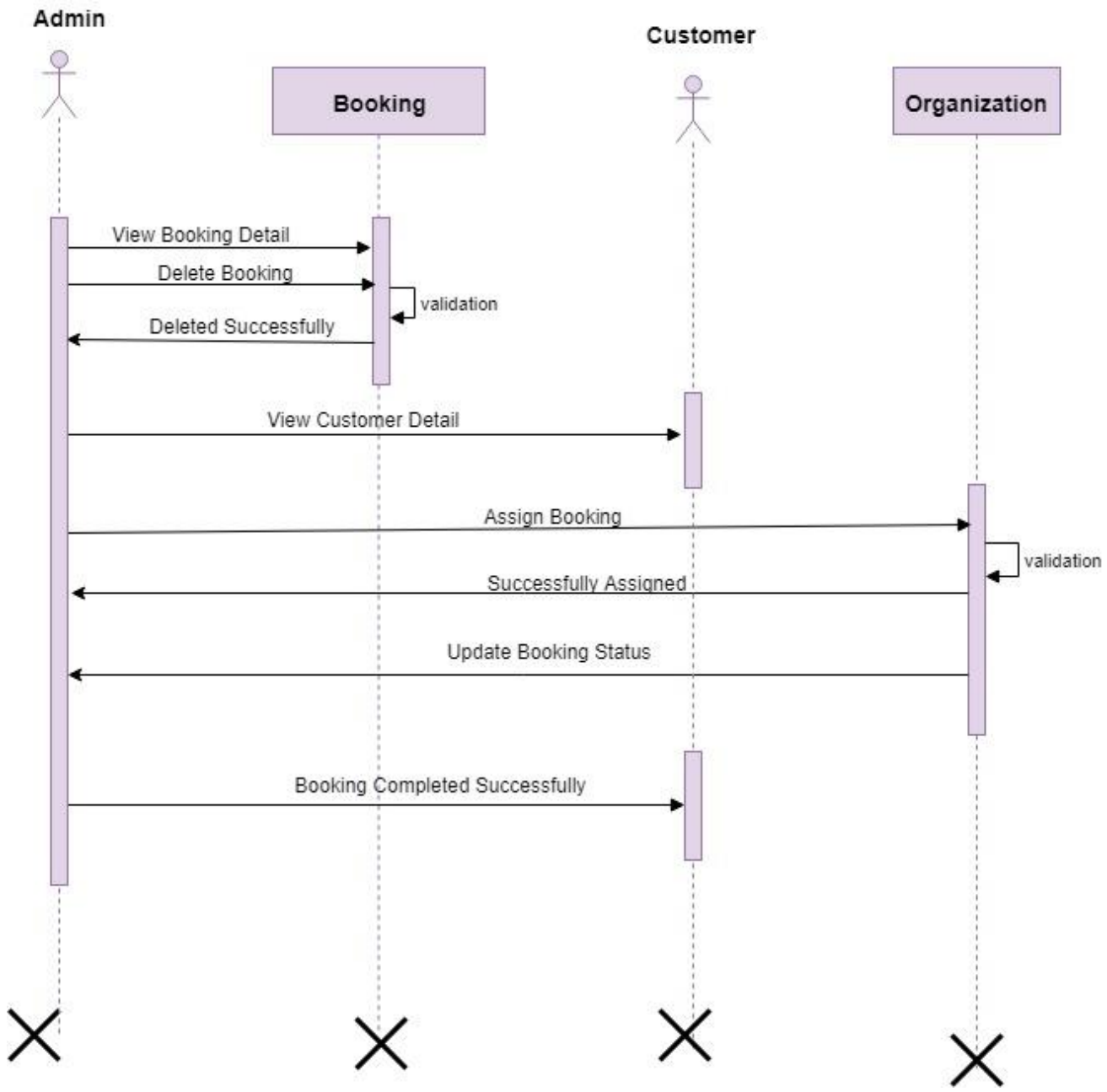


Fig.27 Sequence Diagram 4

Organization login:

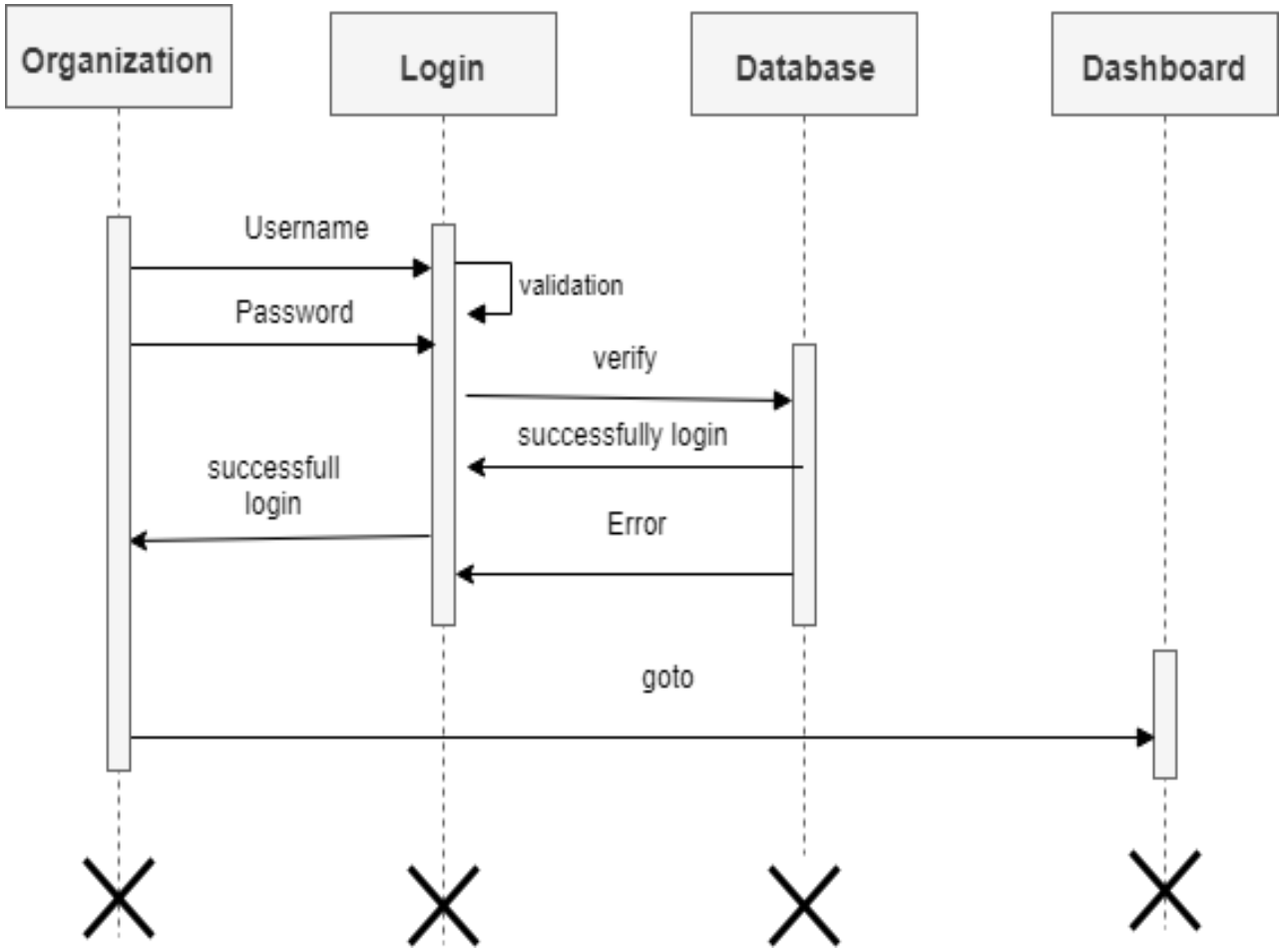


Fig.26 Sequence Diagram 5

Organization Working:

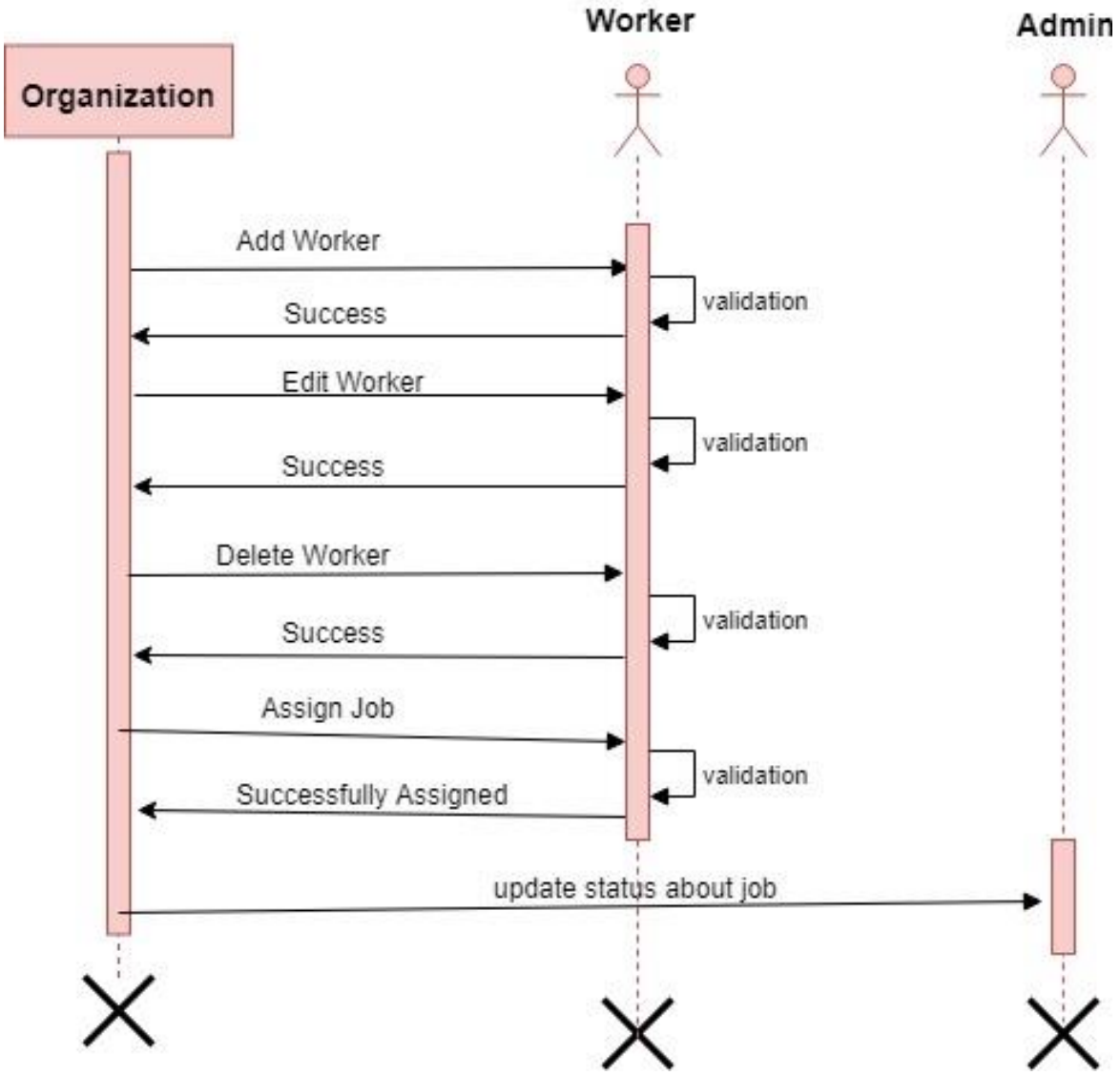


Fig.27 Sequence Diagram 6

Customer registration:

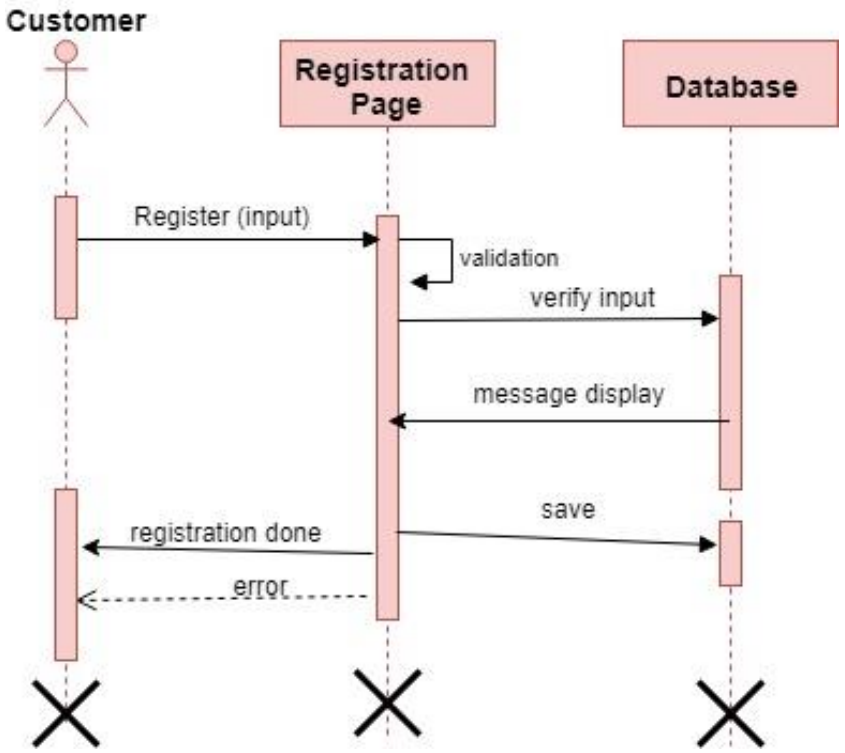


Fig.28 Sequence Diagram 7

Customer Login:

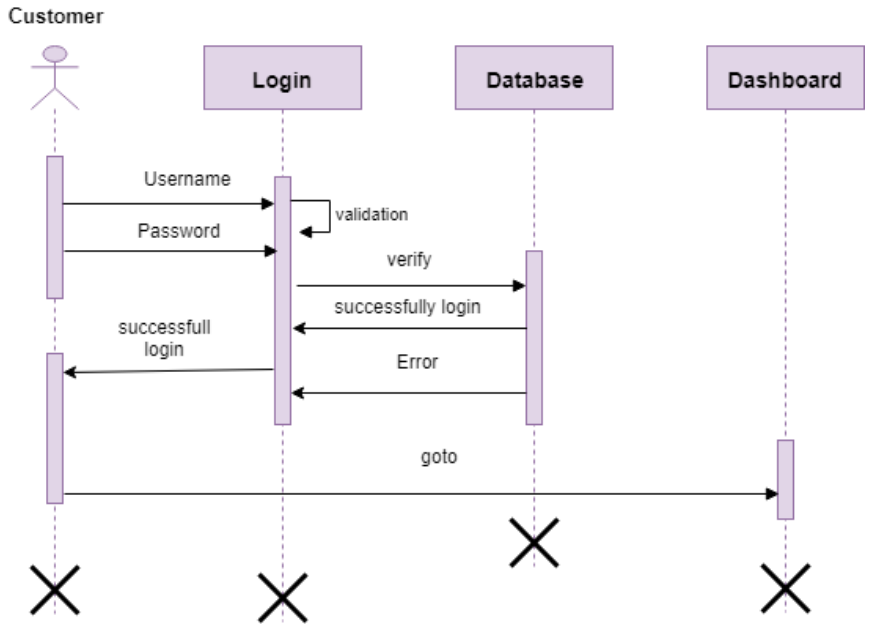


Fig.29 Sequence Diagram 8

Customer Booking:

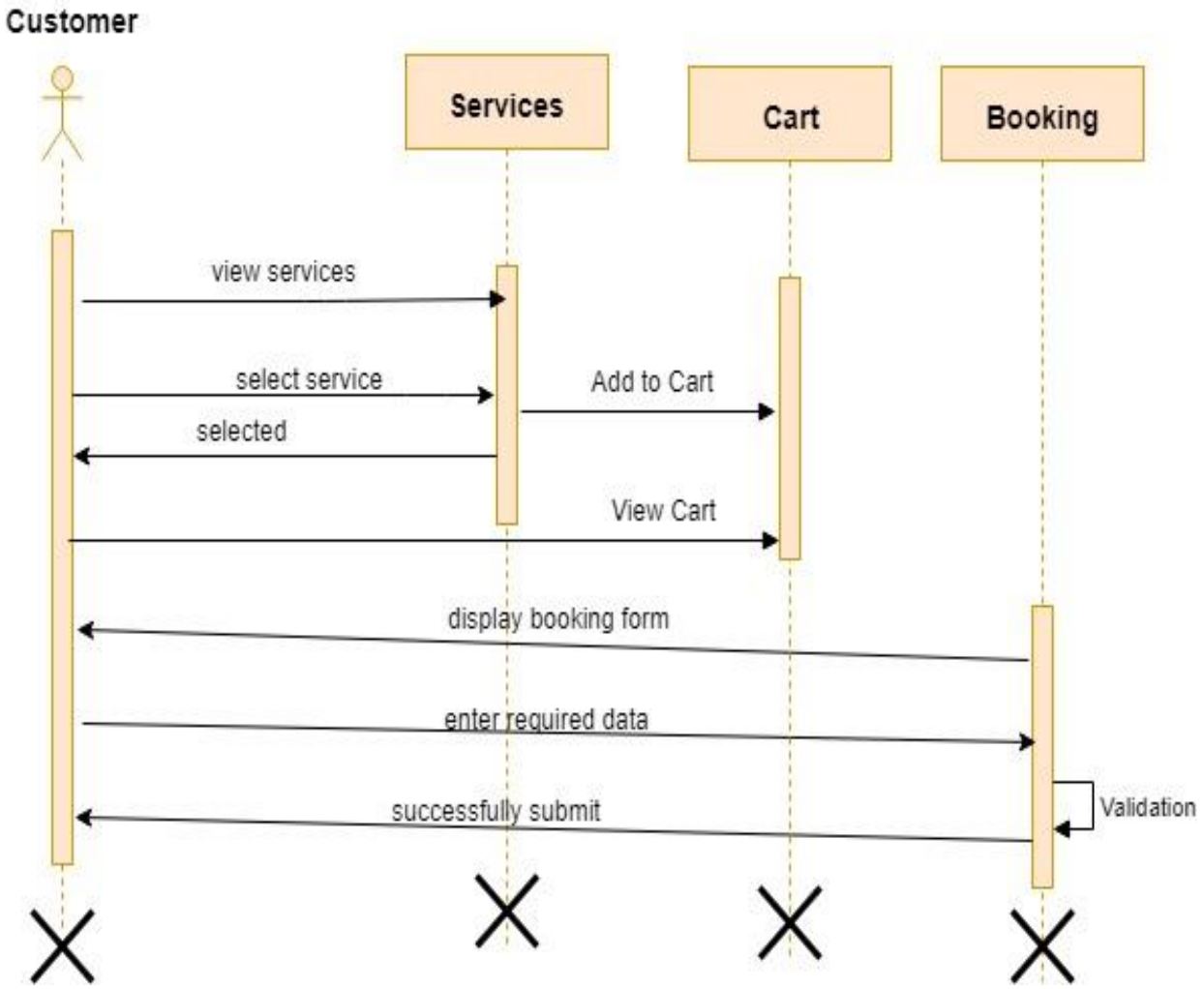


Fig.30 Sequence Diagram 9

7.4. Entity Relationship Diagram:

An entity-relationship diagram, or ERD, is a chart that visually represents the relationship between database entities. ERDs model an organization's data storage requirements with three main components: entities, attributes, and relationships.

- **Entities:**

Entities which represent people, places, items, events, or concepts.

- **Attributes:**

Attributes which represent properties or descriptive qualities of an entity. These are also known as data elements.

- **Relationships:**

Relationships which represent the link between different entities.

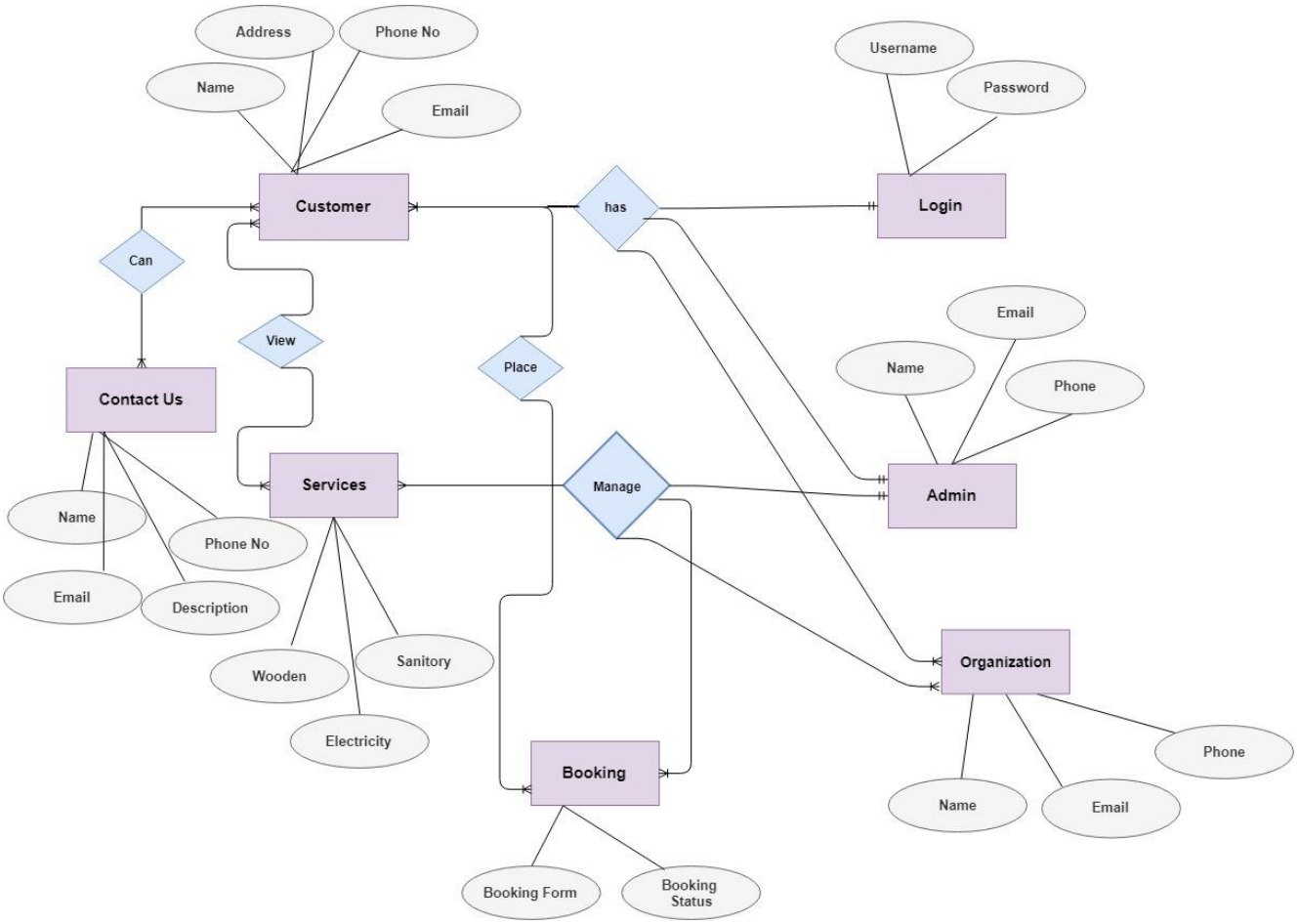


Fig.31 ERD

Appendix C: Design Models

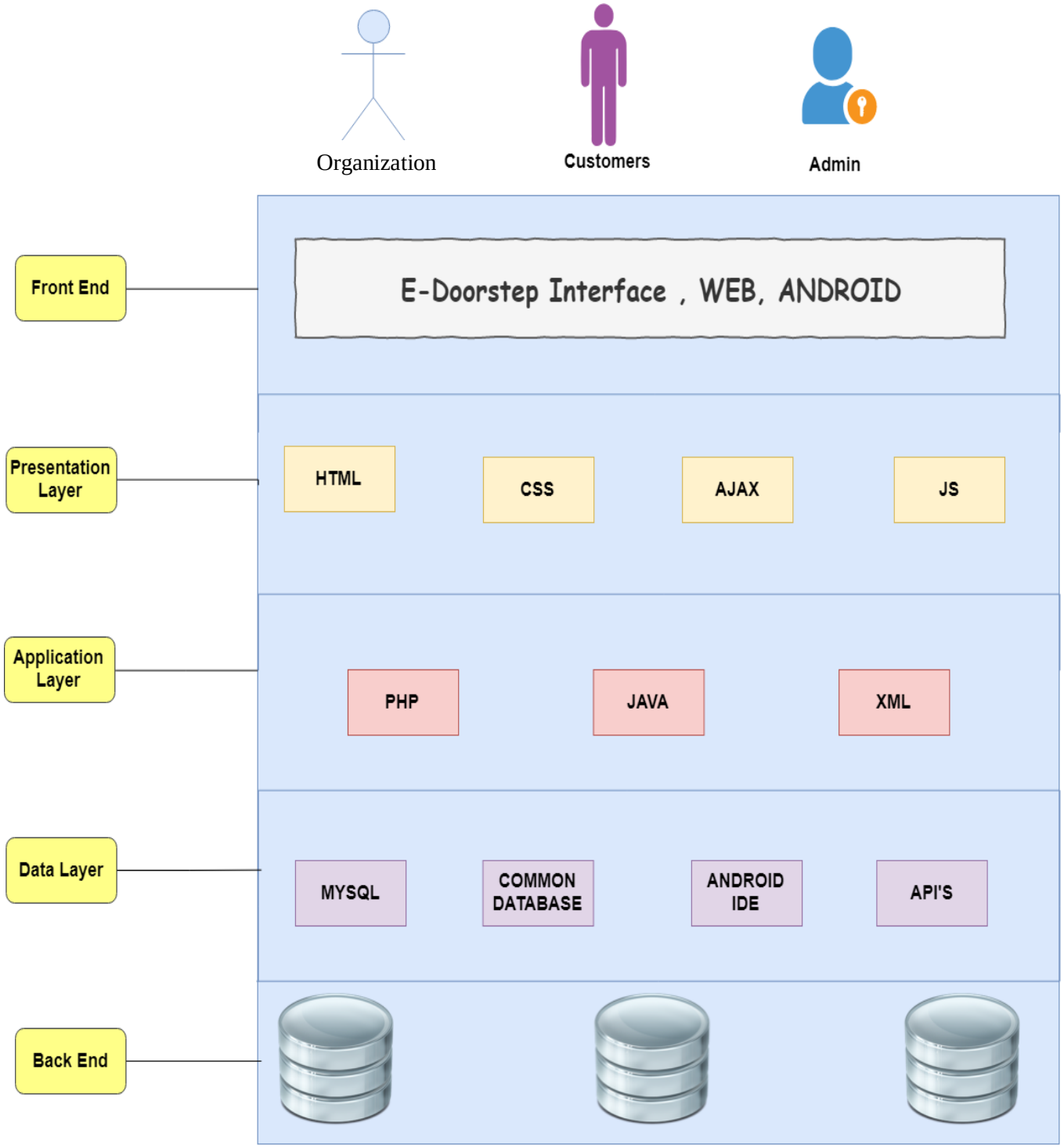


Fig.32 Architecture Diagram

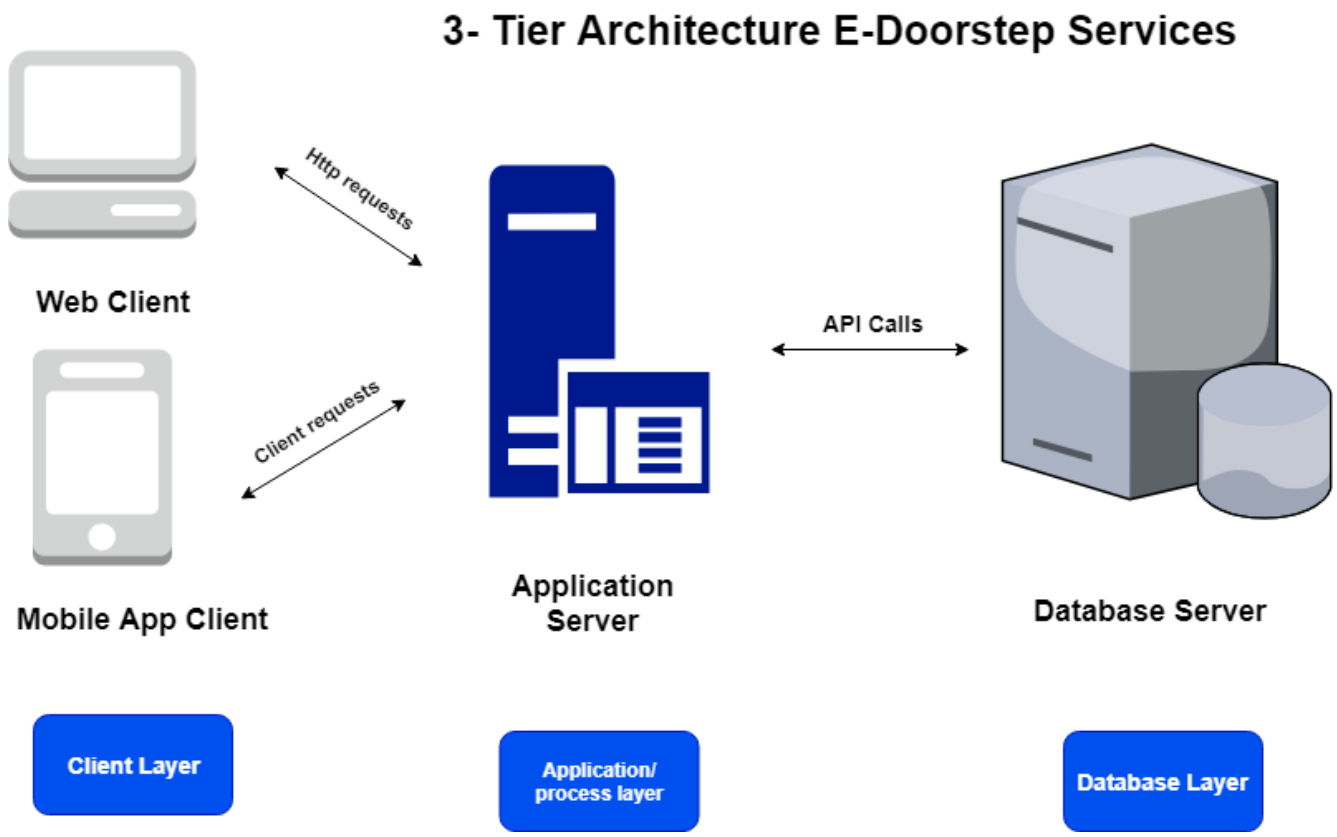
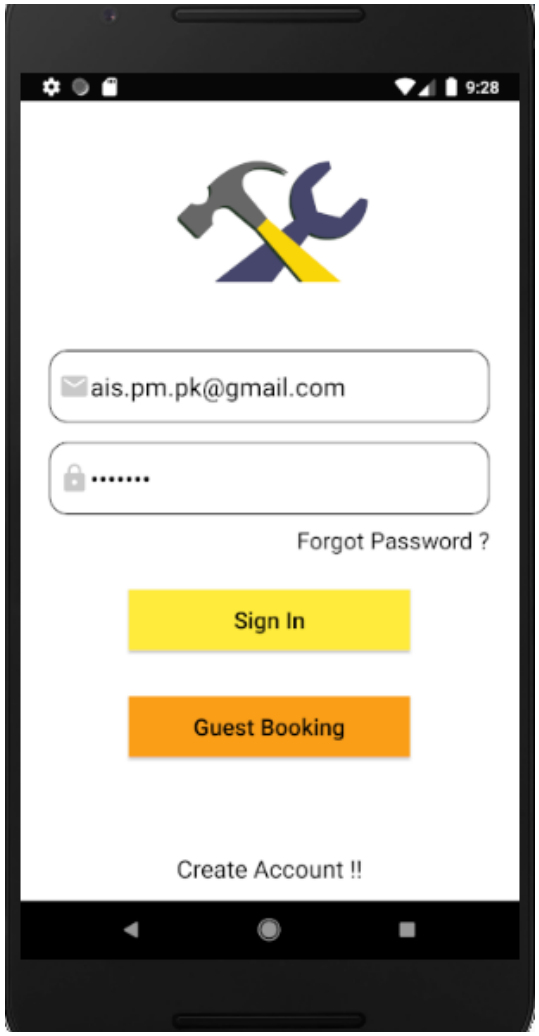


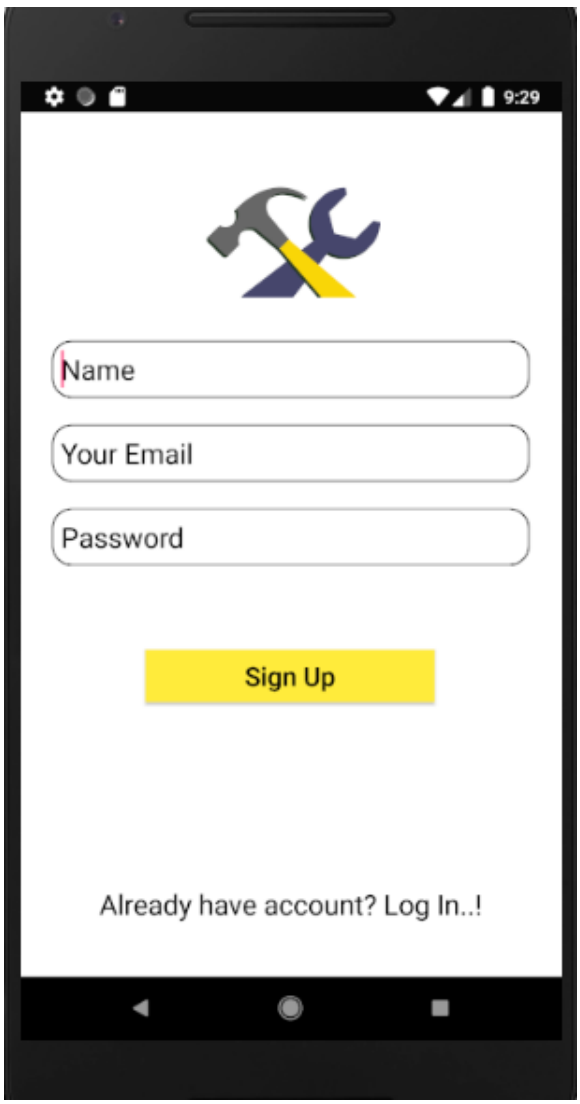
Fig.33 Tier Diagram

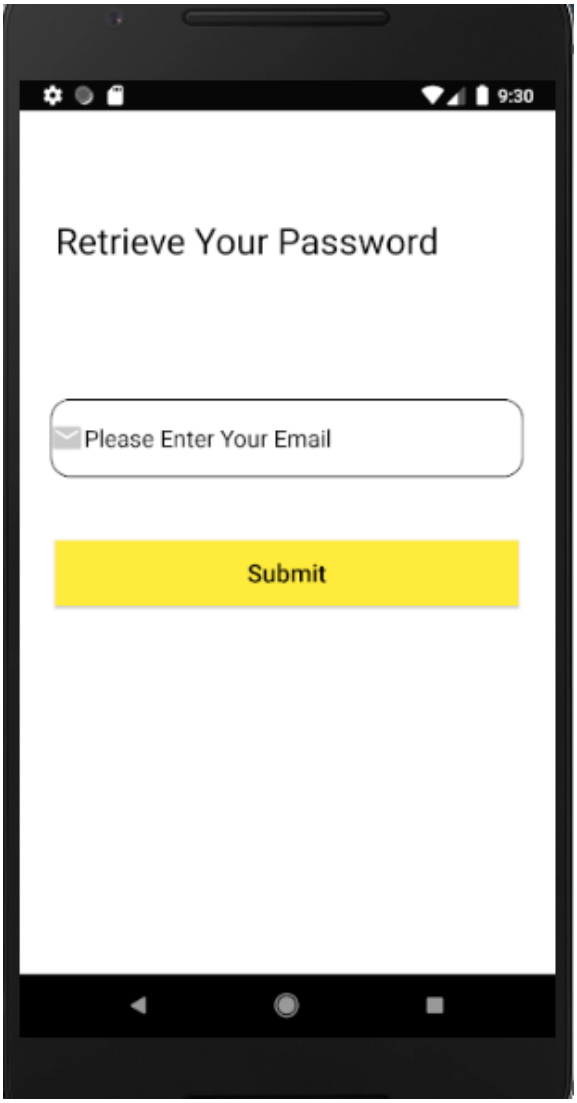
Appendix D: Screenshots

Welcome slides

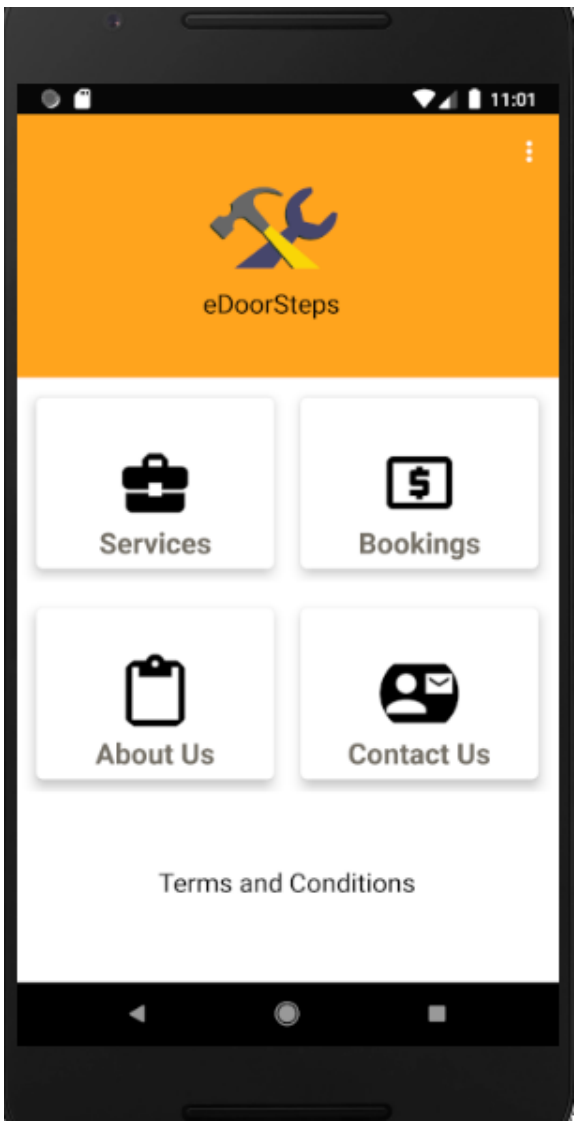


Welcome slide and login Activity

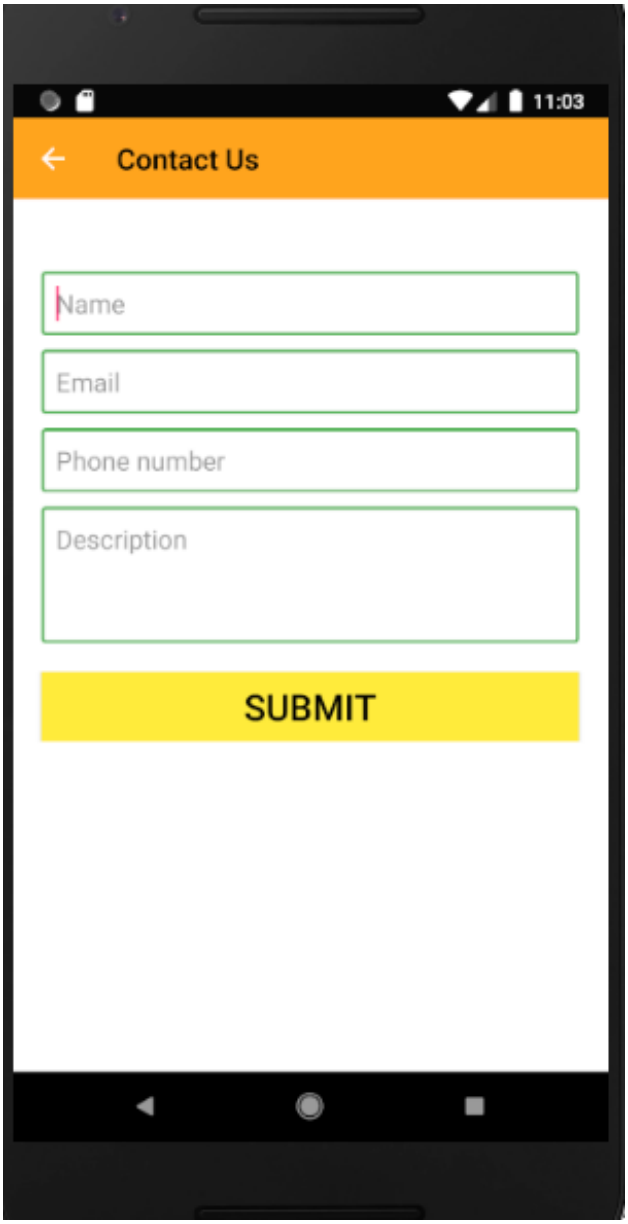




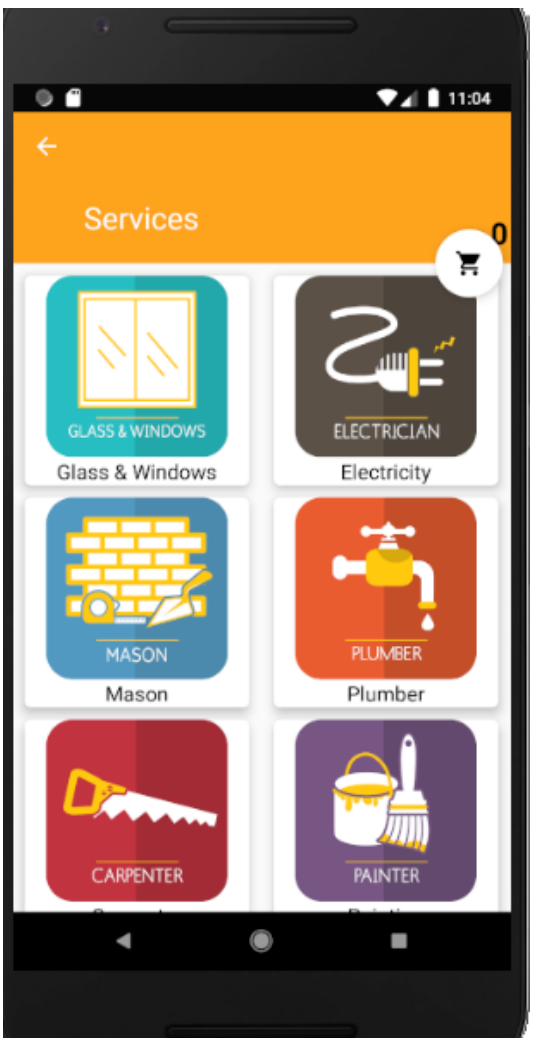
Signup Activity & Password Reset Activity

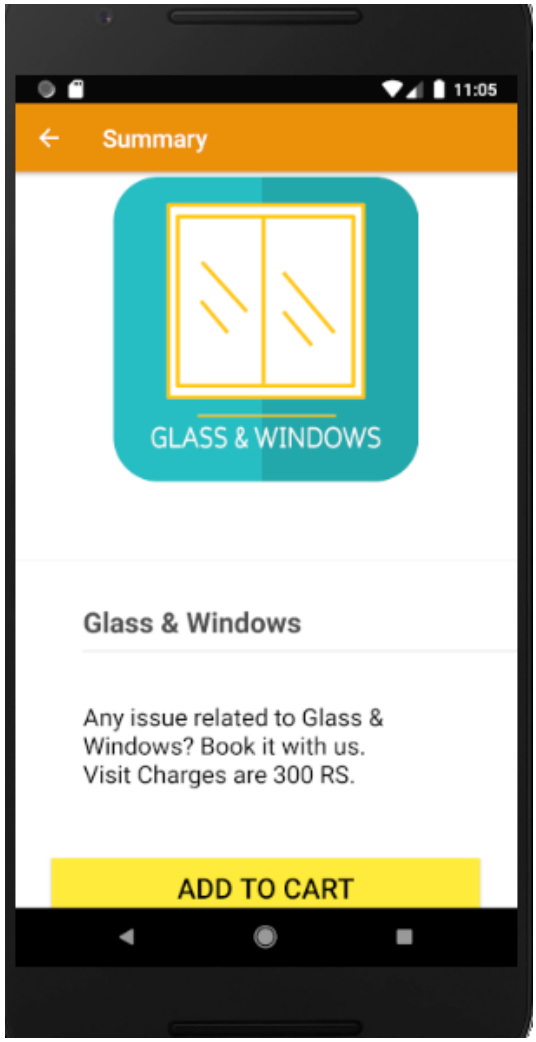


Home Activity & Profile Activity

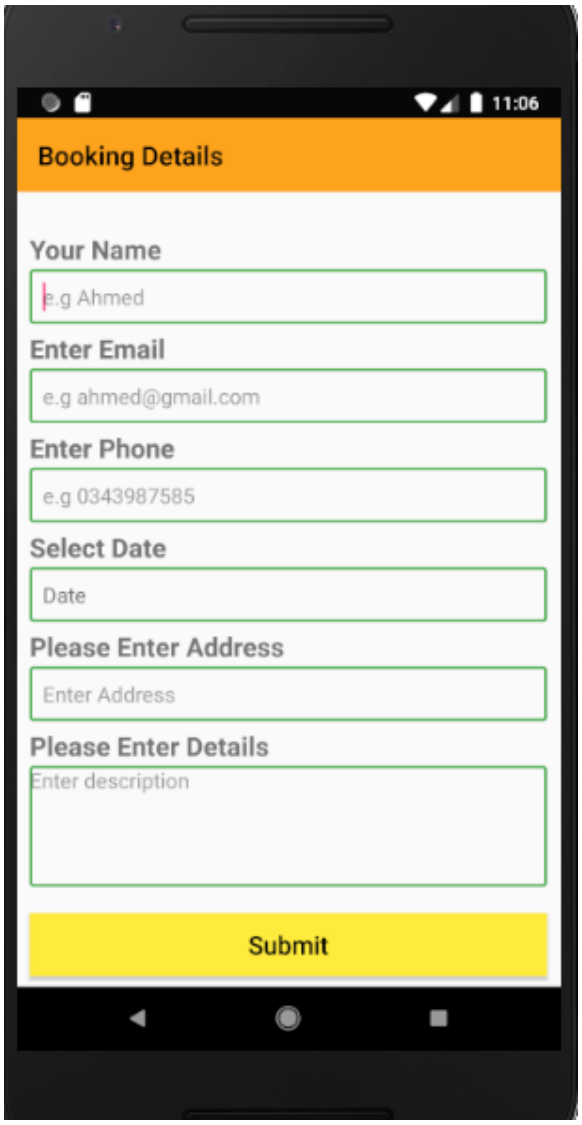


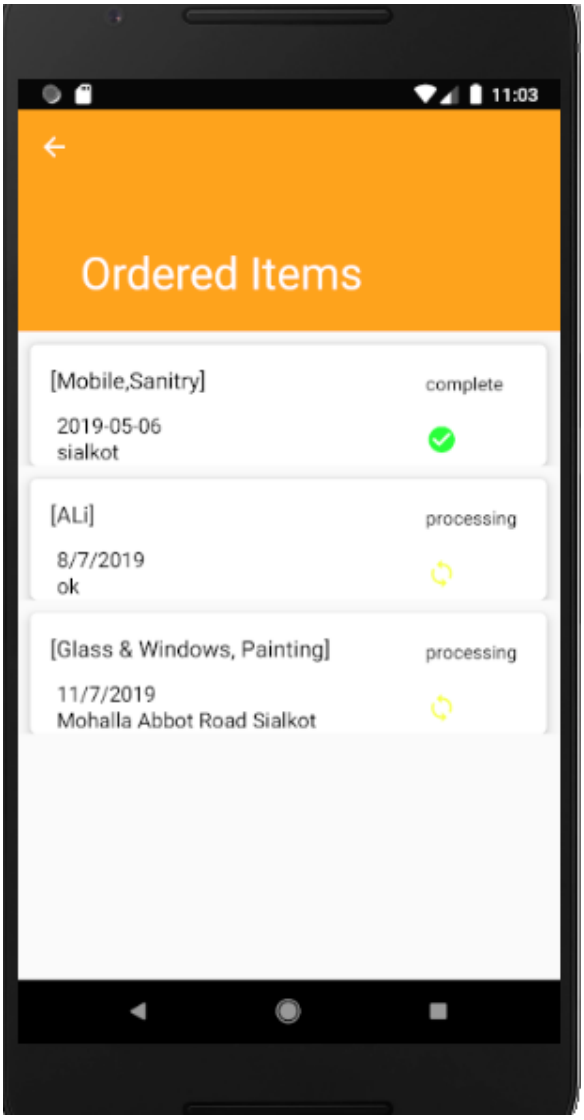
Contact Us Page



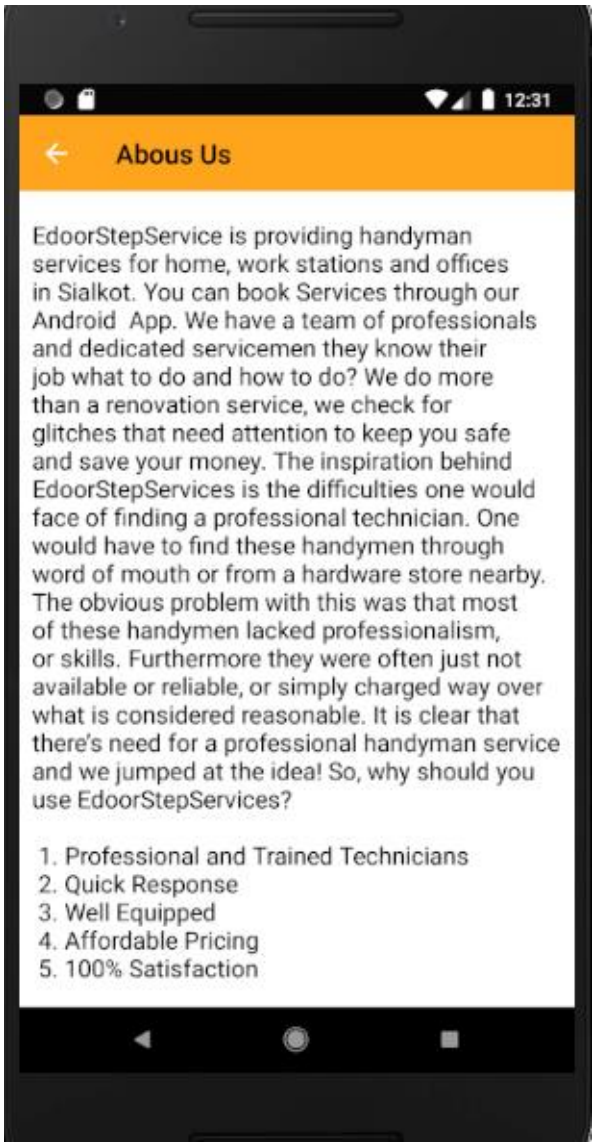


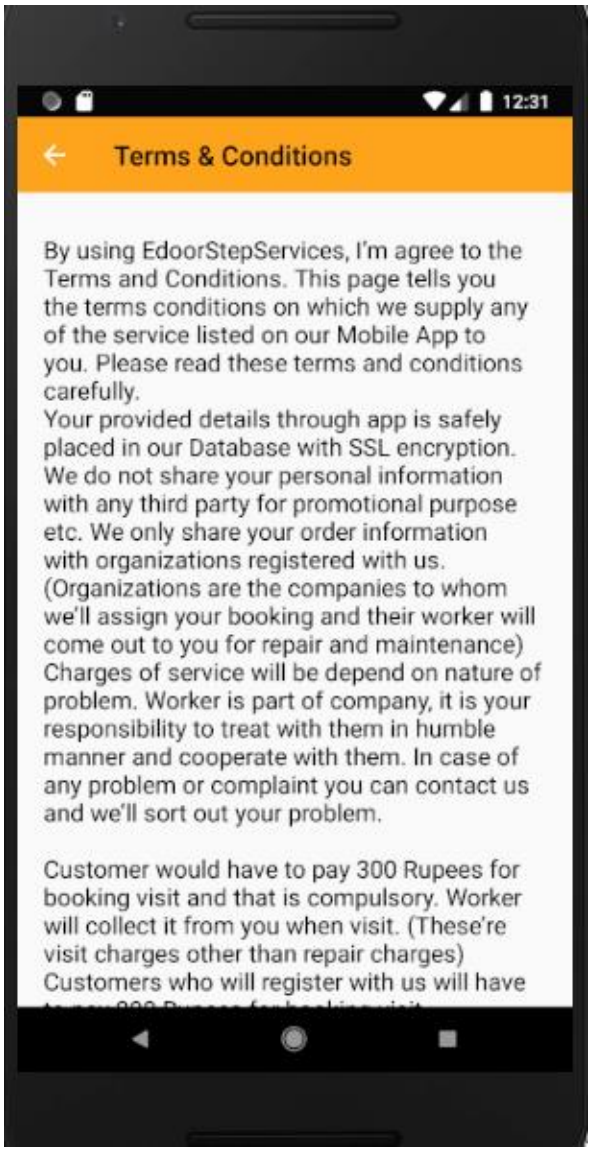
Services & Individual service Activity





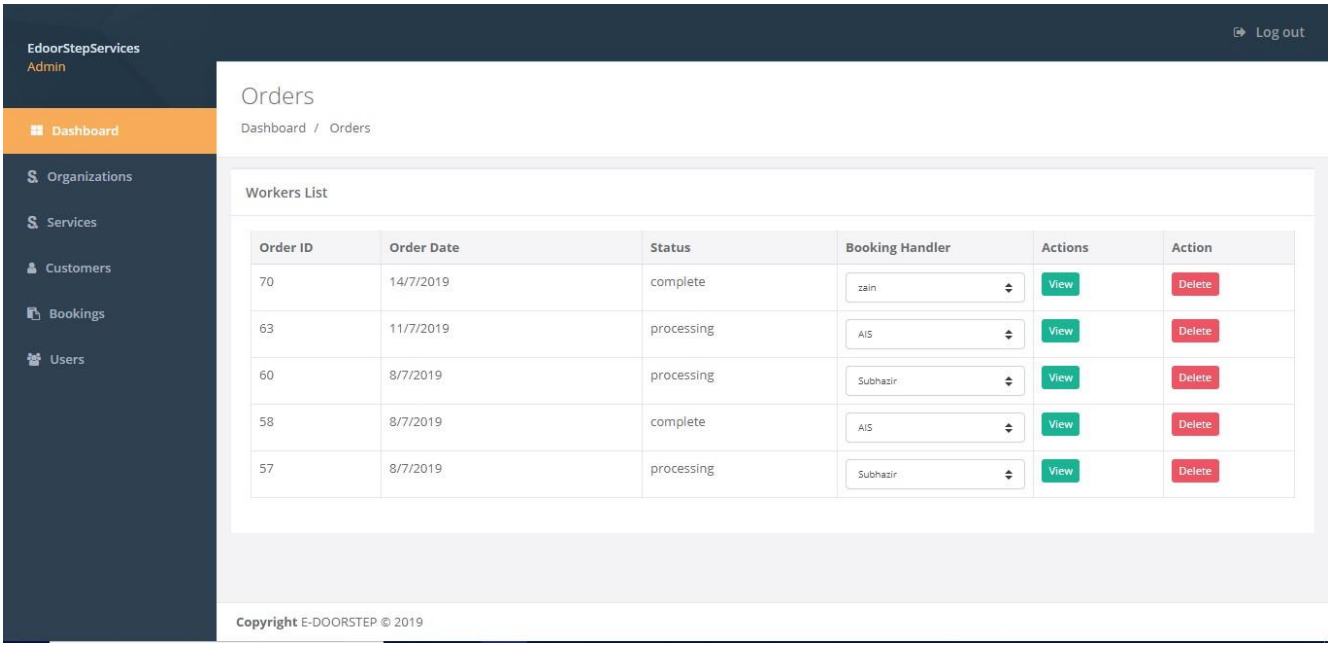
Booking and Ordered items



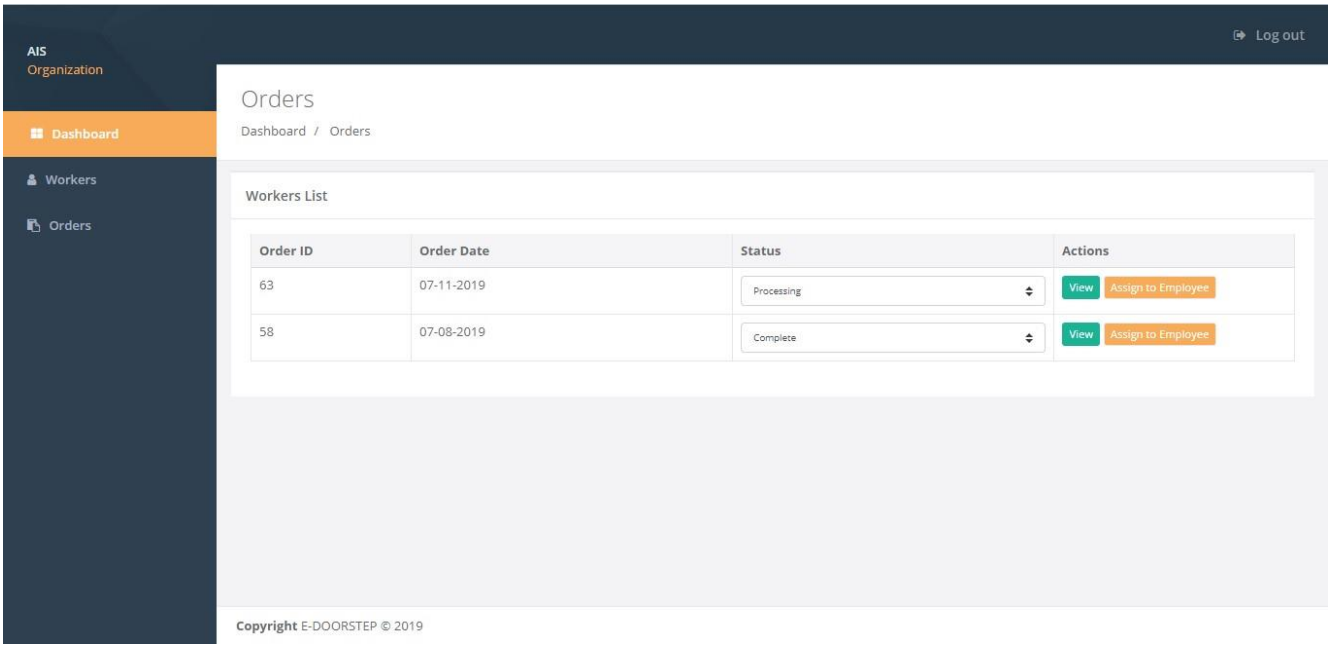


About us and Terms & Condition

Admin Dashboard



Organization Dashboard



Appendix F: IV & V Report

(Independent verification & validation)

IV & V Resource

Name Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
…					

Table 1: List of non-trivial defects